API Reference Manual

Documentaion Serial Number:	
Documentation Classification:	**Confidential**
Documentation Owner:	
Documentation Version:	**1.0**

AICXTEK Confidential

Author:	**R&D Center**
Date:	**2022.8**

本文档（包括任何附件，及其不时的修订）为归芯科技（深圳）有限公司及其关联公司（合称“归芯”）的保密信息。归芯有权不时修订本文档。本文档如有变更，恕不另行通知。
除非已另行获得归芯的明确授权，请勿向第三方透露本文档的任何内容，请勿将本文档的任何内容以任何方式进行复制、发布、修改、传播、分发、翻印、拆解、反向工程、反编译或者为任何目的使用。
归芯始终持有本文档的全部所有权和著作权、专利权、商业秘密、商标权和其他知识产权。除非归芯另行明确授予有关权利，本文档不提供有关上述知识产权的任何权利。
归芯按“现状”提供本文档及归芯产品或服务。归芯未对本文档中的任何内容或本文档涉及的归芯的产品或服务作出任何明示或暗示的陈述、保证或承诺，包括但不限于任何针对特定目的的适销性或适用性，准确性或完整性，或不侵权。
在法律允许的范围内，归芯不对与本文档有关或因本文档内容造成的任何损失承担责任。在任何情况下，归芯都不对因使用本文档造成的或与之有关的任何间接、偶然的、附带或惩罚性损失，或任何利润、收入的损失承担责任。

This document (including any attachment, as amended from time to time) is confidential information of aicxtek technologies co.,ltd. and its affiliates (collectively referred to as “aicxtek”). Aicxtek reserves the right to amend this document from time to time. Any changes to this document will not be notified.
Unless otherwise expressly authorized by aicxtek, you should not disclose any of this document to any third parties. Nor should you copy, publish, modify, broadcast, distribute, photocopy, disassemble, reverse engineer, or decompile any of this document or use it for any purpose.
Aicxtek retains all rights of ownership and copyright, patent rights, trade secrets, trademark rights and other intellectual property rights pertaining to this document. Except for rights otherwise expressly granted by aicxtek, this document does not provide any rights to any of the above intellectual property rights.
Aicxtek provides this document and aicxtek’s products or services on a “as-is” basis. Aicxtek does not provide any representations or warranties on any of this document, or aicxtek’ products or services, whether express or implied, including without limitation, merchantability, fitness for a particular purpose, accuracy or completeness, or non-infringement.
To the extent permitted by law, aicxtek is not liable for any loss relating to or arising from this document. Aicxtek is not in any event be liable for any indirect, incidental or punitive damages, or loss of profits or revenue.

AICXTEK Confidential

Contents

Contents 2
1 Terminologies 15
2 OS 16
Brief instruction 16
2.1 thread 16
2.1.1 aos_create_thread 16
2.1.2 aos_create_thread_simply 16
2.1.3 aos_delete_thread 17
2.1.4 aos_change_thread_pri 17
2.1.5 aos_identify_thread 17
2.1.6 aos_suspend_thread 18
2.1.7 aos_resume_thread 18
2.1.8 aos_terminate_thread 18
2.1.9 aos_is_thread_created 18
2.2 timer 18
2.2.1 aos_create_timer 18
2.2.2 aos_delete_timer 19
2.2.3 aos_activate_timer 19
2.2.4 aos_deactivate_timer 19
2.2.5 aos_change_timer 20
2.2.6 aos_get_remaining_time 20
2.2.7 aos_is_timer_created 20
2.2.8 aos_get_timer_info 20
2.3 mutex 21
2.3.1 aos_create_mutex 21
2.3.2 aos_delete_mutex 21
2.3.3 aos_get_mutex 21
2.3.4 aos_put_mutex 22
2.4 semaphore 22
2.4.1 aos_create_semaphore 22
2.4.2 aos_delete_semaphore 22
2.4.3 aos_get_semaphore 22
2.4.4 aos_put_ semaphore 23
2.5 event 23
2.5.1 aos_create_event 23
2.5.2 aos_delete_event 23
2.5.3 aos_get_event 23

AICXTEK Confidential

2.5.4 aos_set_event 24
2.6 queue 24
2.6.1 aos_create_queue 24
2.6.2 aos_delete_queue 24
2.6.3 aos_recv_from_queue 25
2.6.4 aos_send_to_queue 25
2.6.5 aos_send_to_queue_head 25
2.7 queue 26
2.7.1 aos_send_msg 26
2.7.2 aos_send_msg_head 26
2.7.3 aos_recv_msg 26
2.8 memory 27
2.8.1 aos_malloc 27
2.8.2 aos_zalloc 27
2.8.3 aos_dma_malloc 27
2.8.4 aos_free 28
2.8.5 aos_dma_free 28
2.8.6 aos_is_heap_ptr 28
2.9 Interrupt 28
2.9.1 AOS_INTERRUPT_DECLARE 29
2.9.2 AOS_INTERRUPT_DISABLE 29
2.9.3 AOS_INTERRUPT_RESTORE 29
2.9.4 aos_request_irq 29
2.9.5 aos_request_irq_with_half_bottom 29
2.9.6 aos_release_irq 30
2.10 time 30
2.10.1 aos_get_tickcount 30
2.10.2 aos_get_systime 30
2.10.3 aos_set_systime 31
2.10.4 aos_time_to_tick 31
2.10.5 aos_tick_to_time 31
2.10.6 aos_sleep 31
2.11 debug 31
2.11.1 AOS_ASSERT 31
2.11.2 AOS_ASSERT_INFO 32
3 Auto init 32
Brief instruction 32
3.1 AIC_XXX_INIT_EXPORT 32
4 ALOG 32
Brief instruction 32
4.1 alog_xxx 33
5 DIAG 33
Brief instruction 33
5.1 diag_register_cmd_handler 33

AICXTEK Confidential

5.2 diag_cmd_response 34
6 Network 34
6.1 HTTP and HTTPS client common api 34
Briefly instruction 34
6.1.1 aic_process_https_url 34
6.1.2 aic_https_store_redir_info 35
6.1.3 aic_https_client_set_cid 35
6.1.4 aic_https_client_content_type_header_add 35
6.1.5 aic_https_client_response_body_get 36
6.1.6 aic_https_client_response_data_get 36
6.1.7 aic_https_client_request_packet_allocate 36
6.1.8 aic_https_client_put_packet 37
6.1.9 aic_https_client_put_data 37
6.1.10 aic_https_client_request_chunked_set 37
6.1.11 aic_https_client_request_header_add 38
6.1.12 aic_https_client_request_initialize 38
6.1.13 aic_https_client_request_send 38
6.1.14 aic_https_client_response_header_callback_set 39
6.1.15 aic_https_client_total_transfer_bytes_get 39
6.1.16 aic_https_client_actual_bytes_transferred_get 39
6.1.17 aic_https_client_total_receive_bytes_get 40
6.1.18 aic_https_client_actual_bytes_received_get 40
6.2 Hypertext Transfer Protocol Secure (HTTPS using TLS) Client 40
Briefly instruction 40
6.2.1 aic_https_client_create 40
6.2.2 aic_https_client_delete 41
6.2.3 aic_https_ca_crt_add 41
6.2.4 aic_https_ca_crt_remove 41
6.2.5 aic_https_client_crt_add 41
6.2.6 aic_https_client_crt_remove 42
6.2.7 aic_https_client_get_start 42
6.2.8 aic_https_client_range_get_start 43
6.2.9 aic_https_client_post_start 44
6.2.10 aic_https_client_post_start_ext 44
6.2.11 aic_https_client_put_start 45
6.2.12 aic_https_client_head_start 45
6.2.13 aic_https_client_delete_start 45
6.2.14 aic_https_client_secure_connect 46
6.3 Hypertext Transfer Protocol Secure (HTTPS using TLS) Server 46
Briefly instruction 46
6.3.1 aic_https_server_create 46
6.3.2 aic_https_server_delete 47
6.3.3 aic_https_server_cache_info_callback_set 47
6.3.4 aic_https_server_callback_data_send 47

AICXTEK Confidential

6.3.5 aic_https_server_callback_generate_response_header 48
6.3.6 aic_https_server_callback_packet_send 48
6.3.7 aic_https_server_callback_response_send 48
6.3.8 aic_https_server_content_get 49
6.3.9 aic_https_server_content_length_get 49
6.3.10 aic_https_server_get_entity_content 50
6.3.11 aic_https_server_get_entity_header 50
6.3.12 aic_https_server_gmt_callback_set 50
6.3.13 aic_https_server_invalid_userpassword_notify_set 51
6.3.14 aic_https_server_mime_maps_additional_set 51
6.3.15 aic_https_server_response_packet_allocate 51
6.3.16 aic_https_server_packet_content_find 52
6.3.17 aic_https_server_packet_get 52
6.3.18 aic_https_server_param_get 52
6.3.19 aic_https_server_query_get 53
6.3.20 aic_https_server_response_chunked_set 53
6.3.21 aic_https_server_secure_configure 54
6.3.22 aic_https_server_start 54
6.3.23 aic_https_server_stop 54
6.3.24 aic_https_server_type_get 54
6.3.25 aic_https_server_digest_authenticate_notify_set 55
6.3.26 aic_https_server_authentication_check_set 55
6.4 Hypertext Transfer Protocol (HTTP) Client 55
Briefly instruction 55
6.4.1 aic_http_client_create 56
6.4.2 aic_http_client_delete 56
6.4.3 aic_http_client_get_start 56
6.4.4 aic_http_client_range_get_start 56
6.4.5 aic_http_client_post_start 57
6.4.6 aic_http_client_post_start_ext 58
6.4.7 aic_http_client_put_start 58
6.4.8 aic_http_client_head_start 59
6.4.9 aic_http_client_delete_start 59
6.4.10 aic_http_client_connect 59
6.5 BSD Socket 60
Briefly instruction 60
6.5.1 socket 60
6.5.2 connect 60
6.5.3 bind 61
6.5.4 listen 61
6.5.5 accept 61
6.5.6 send 62
6.5.7 sendto 62
6.5.8 recv 63

AICXTEK Confidential

6.5.9 recvfrom 63
6.5.10 soc_close 64
6.5.11 getaddrinfo 64
6.5.12 aic_socket_register_establish_notify 64
6.5.13 aic_socket_unregister_establish_notify 65
6.5.14 aic_tcp_socket_retransmit_configure 65
6.5.15 aic_tcp_socket_transmit_max_queue_configure 65
6.5.16 aic_socket_register_connect_notify 66
6.5.17 aic_socket_unregister_connect_notify 66
6.5.18 aic_socket_register_receive_notify 66
6.5.19 aic_socket_unregister_receive_notify 67
6.5.20 aic_socket_register_disconnect_notify 67
6.5.21 aic_socket_unregister_disconnect_notify 67
6.5.22 aic_socket_register_receive_ext_notify 67
6.5.23 aic_socket_unregister_receive_ext_notify 68
6.5.24 aic_socket_register_tcp_ack_notify 68
6.5.25 aic_socket_unregister_tcp_ack_notify 68
6.5.26 aic_socket_register_udp_drop_notify 68
6.5.27 aic_socket_unregister_udp_drop_notify 69
6.5.28 aic_gethostbyname 69
6.6 AT command process api 69
Brief instruction 69
6.6.1 aic_add_at_cmd_handle 69
6.6.2 aic_add_at_resp_handle 70
6.6.3 aic_add_at_urc_handle 70
6.6.4 aic_register_cp_urc_callback 70
6.6.5 aic_unregister_cp_urc_callback 71
6.6.6 aic_assemble_and_handle_at_cmd 71
6.6.7 aic_send_cmd_with_callback 71
6.6.8 aic_send_cmd_and_wait 72
6.6.9 aic_fill_transmit_cmd_resp 72
6.6.10 aic_reply_error_resp 73
6.6.11 aic_reply_ok_resp 73
6.6.12 aic_transmit_at_data 73
6.6.13 aic_transmit_at_cmd_urc 74
6.6.14 aic_get_each_line_from_resp 74
6.6.15 aic_split_paras_to_str_array 74
6.6.16 aic_set_at_handle_mode 75
6.6.17 aic_free_paras_mem 75
6.7 Transport Layer Security (TLS) 76
6.7.1 aic_create_tls_socket 76
6.7.2 aic_close_tls_socket 76
6.7.3 aic_tls_ca_certificate_add 76
6.7.4 aic_tls_ca_certificate_remove 77

AICXTEK Confidential

6.7.5 aic_tls_local_certificate_add 77
6.7.6 aic_tls_local_certificate_remove 77
6.7.7 aic_tls_ciphersuite_set 78
6.7.8 aic_tls_session_protocol_version_override 78
6.7.9 aic_secure_tls_session_renegotiate 78
6.7.10 aic_secure_tls_session_time_function_set 78
6.7.11 aic_tls_set_session_sni_extension 79
6.8 Domain Name System(DNS) 79
Brief instruction 79
6.8.1 aic_clear_dns_cache 79
6.9 Access Point Name(APN) 79
Brief instruction 80
6.9.1 aic_add_additional_apn 80
6.9.2 aic_get_fixed_apn_by_plmn 80
6.9.3 aic_get_additional_apn_by_plmn 80
6.9.4 aic_get_apn_name_by_plmn 81
6.9.5 aic_get_apn_auth_type 81
7 NV 82
Brief instruction 82
7.1 aic_nvram_get_length 82
7.2 aic_nvram_read 82
7.3 aic_nvram_write 82
7.4 aic_nvram_sync_write 83
7.5 aic_nvram_flush 83
7.6 aic_nvram_restore_factory 83
8 Miscdata 83
Brief instruction 83
8.1 aic_md_get_length 84
8.2 aic_md_read 84
8.3 aic_md_write 84
8.4 aic_md_delete 84
8.5 aic_md_restore_factory 85
9 LCD 85
Brief instruction 85
9.1 display_refresh_sync 85
9.2 display_refresh 85
9.3 display_suspend 85
9.4 display_resume 86
9.5 aic_bl_brightness_set 86
9.6 aic_bl_brightness_get 86
9.7 register_backlight_switch_monitor 86
9.8 unregister_backlight_switch_monitor 86
10 Camera 87

AICXTEK Confidential

Brief instruction....87
10.1 Camera configure files....87
10.1.1 Header files....87
10.1.1.1 camera_fb_cfg.h....87
10.1.1.2 camera_cfg.h....87
10.2 Camera hal....88
10.2.1 camera_open....88
10.2.2 camera_close....88
10.2.3 camera_preview....89
10.2.4 camera_capture_prepare....89
10.2.5 Camera_capture....89
10.2.6 camera_set_fps....89
10.2.7 camera_get_fps....90
10.2.8 camera_ioctl....90
10.3 Jpeg interface....90
10.3.1 jpeg_compress_struct....90
10.3.2 struct_decompress_struct....90
10.3.3 jpeg_create_compress....91
10.3.4 jpeg_std_error....91
10.3.5 jpeg_stdio_dest....91
10.3.6 jpeg_mem_dest....91
10.3.7 jpeg_set_defaults....91
10.3.8 jpeg_start_compress....92
10.3.9 jpeg_write_raw_data....92
10.3.10 jpeg_write_scanlines....92
10.3.11 jpeg_finish_compress....93
10.3.12 jpeg_destroy_compress (j_compress_ptr cinfo)....93
10.3.13 jpeg_abort_compress....93
10.3.14 jpeg_create_decompress....93
10.3.15 jpeg_stdio_src....93
10.3.16 jpeg_read_header....93
10.3.17 jpeg_start_decompress....94
10.3.18 Jpeg_read_scanlines....94
10.3.19 Jpeg_finish_decompress....94
10.3.20 jpeg_destroy_decompress....95
10.3.21 jpeg_abort_decompress....95
11 Audio....95
Brief instruction....95
11.1 enum astream_type....95
11.2 enum aplay_type....96
11.3 enum astream_format....96
11.4 enum astream_source....97
11.5 enum astream_channel....97
11.6 enum astream_direct....97

AICXTEK Confidential

11.7 enum audio_mode 97
11.8 enum audio_device 98
11.9 enum audio_output_flag 98
11.10 enum astream_cb_event 99
11.11 enum tone_type 99
11.12 astream_user_cb 100
11.13 audio_open_output 100
11.14 audio_open_input 100
11.15 audio_open 101
11.16 audio_play 101
11.17 audio_stop 101
11.18 audio_close 102
11.19 audio_play_buf 102
11.20 audio_play_file 102
11.21 audio_play_stream 103
11.22 audio_play_tone 103
11.23 audio_pause 103
11.24 audio_resume 103
11.25 audio_read 104
11.26 audio_write 104
11.27 audio_drain 104
11.28 audio_force_out_device 105
11.29 audio_force_in_device 105
11.30 audio_get_device 105
11.31 audio_get_force_device 105
11.32 audio_set_volume 105
11.33 audio_get_volume 106
11.34 audio_set_master_volume 106
11.35 audio_get_master_volume 106
11.36 audio_mute 106
11.37 audio_unmute 106
11.38 audio_mute_all 107
11.39 audio_unmute_all 107
12 Cache 107
Brief instruction 107
Header files 107
12.1 cache_enable 107
12.2 cache_disable 107
12.3 cache_invcc_all 108
12.4 icache_enable 108
12.5 icache_disable 108
12.6 icache_inv_by_range 108
12.7 icache_inv_all 109

AICXTEK Confidential

12.8 dcache_enable 109
12.9 dcache_disable 109
12.10 dcache_clean_by_range 109
12.11 dcache_clean_all 110
12.12 dcache_inv_by_range 110
12.13 dcache_inv_all 110
12.14 dcache_invcc_by_range 110
12.15 dcache_invcc_all 111
13 MPU 111
Brief instruction 111
Enum 111
Struct 112
Header files 112
13.1 mpu_dynamic_conf_region 112
13.2 mpu_dynamic_unconf_region 112
14 DMA 113
Brief instruction 113
14.1 aic_dma_chn_request 113
14.2 aic_dma_chn_free 113
14.3 aic_dma_ll_cfg 113
14.4 aic_dma_chn_cfg 114
14.5 aic_dma_chn_start 114
14.6 aic_dma_chn_get_src 114
14.7 aic_dma_chn_get_dst 114
15 Clock 115
Briefly instruction 115
15.1 clk_set_parent_mux 115
15.2 clk_set_parent_comp 115
15.3 clk_set_rate_comp 116
15.4 clk_set_rate_div 116
15.5 clk_enable 116
15.6 clk_disable 116
15.7 clk_get_parent_mux 116
15.8 clk_get_parent_comp 117
15.9 clk_get_rate_comp 117
15.10 clk_get_rate_mux 117
15.11 clk_get_rate_div 117
16 IPU 117
Brief instruction 117
16.1 IPU_IRQEnable 118
16.2 IPU_IRQDisable 118
17 EIPU 118
Brief instruction 118

AICXTEK Confidential

17.1 EIPU_RegCallback 118
17.2 EIPU_SetPol 119
17.3 EIPU_GetIndata 119
18 Timer 119
Brief instruction 119
18.1 aic_get_tick 119
18.2 aic_get_timestamp 120
18.3 aic_delay 120
18.4 aic_mdelay 120
18.5 aic_udelay 120
19 UART 120
Brief instruction 120
19.1 uart_open 121
19.2 uart_close 121
19.3 uart_read 121
19.4 uart_write 121
19.5 uart_set_baud_rate 122
20 I2C 122
Brief instruction 122
20.1 i2c_open 122
20.2 i2c_close 122
20.3 i2c_master_send 123
20.4 i2c_master_recv 123
20.5 i2c_master_send_then_recv 123
20.6 i2c_transfer 124
21 PMIC interface 124
Brief instruction 124
21.1 pmic_writel 124
21.2 pmic_readb 125
21.3 pmic_readl 125
21.4 pmic_burst_write 125
21.5 pmic_burst_read 125
22 SSP 126
Brief instruction 126
22.1 ssp_device_register 127
22.2 ssp_read 127
22.3 ssp_write 128
22.4 ssp_write_read 128
22.5 ssp_write_then_read 128
22.6 ssp_write_then_write 129
23 USB 129
23.1 usb_get_connect_status 129
24 GPIO 129

AICXTEK Confidential

Brief instruction....129
24.1 aic_gpio_output....130
24.2 aic_gpio_input....130
24.3 aic_gpio_level_get....130
25 GPO....133
Brief instruction....133
25.1 aic_gpo_output....133
25.2 aic_gpo_get_level....133
25.3 aic_gpo_pin_bare_cfg....134
26 Pin control....134
Brief instruction....134
Header files....134
27 Keypad....134
Briefly instruction....134
27.1 kpc_msg....135
27.2 kpc_bisr_thrd_pid....135
28 GPD....135
Brief instruction....135
28.1 aic_gpd_update....135
28.2 aic_gpd_mode_cfg....135
28.3 aic_gpd_cnt_cfg....136
28.4 aic_gpd_get_cnt....136
28.5 aic_gpd_get_cycle....136
29 PWM....136
Brief instruction....136
29.1 pwm_config....136
30 Watchdog....137
Brief instruction....137
30.1 aic_watchdog_ping....137
30.2 aic_watchdog_start....137
30.3 aic_watchdog_stop....137
30.4 aic_watchdog_set_timeout....138
30.5 aic_watchdog_mode_config....138
31 Power control....138
Brief instruction....138
31.1 aic_get_wakelock....138
31.2 aic_put_wakelock....139
31.3 slp_register_sleep_hook....139
31.4 slp_unregister_sleep_hook....139
31.5 sysoff_shutdown....139
31.6 sysoff_reset....139
31.7 aic_boot_mode_get....140
31.8 sysoff_register_hook....140

AICXTEK Confidential

32 Efuse 140
Brief instruction 140
32.1 aic_efuse_block_write 140
32.2 aic_efuse_block_read 141
33 Flash managment device 141
Brief instruction 141
33.1 fmd_get_partition_name 141
33.2 fmd_open 141
33.3 fmd_close 142
33.4 fmd_read 142
33.5 fmd_write 142
33.6 fmd_erase 143
33.7 fmd_get_partition_size 143
33.8 fmd_get_erase_size 143
34 Flash translation layer 143
Brief instruction 143
34.1 ftlblock_open 144
34.2 ftlblock_close 144
34.3 ftlblock_read 144
34.4 ftlblock_write 144
34.5 ftlblock_get_sector_size 145
34.6 ftlblock_get_sector_num 145
34.7 ftlblock_get_partition_size 145
34.8 ftlrandom_open 145
34.9 ftlrandom_close 146
34.10 ftlrandom_read 146
34.11 ftlrandom_write 146
34.12 ftlrandom_get_sector_size 147
34.13 ftlrandom_get_sector_num 147
34.14 ftlrandom_get_partition_size 147
35 PowerSupply 147
Brief instruction 147
35.1 aic_battery_monitor_start 147
35.2 aic_battery_monitor_stop 148
35.3 aic_pmic_charger_enable 148
35.4 get_psy_info 148
35.5 psy_changed_notifier_register 148
35.6 psy_changed_notifier_unregister 149
36 LED 149
37 Lamp 149
Brief instruction 149
37.1 lamp_enabled 149
37.2 lamp_is_enabled 149

AICXTEK Confidential

37.3 lamp_set_current 149
37.4 lamp_get_current 150
38 Can 150
Brief instruction 150
38.1 can_init 150
38.2 can_recv 150
38.3 can_send 151
38.4 can_send_high_priority 151
38.5 can_set_filter 151
38.6 can_set_bit_timing 151

AICXTEK Confidential

1 Terminologies

AICXTEK Confidential

2 OS

Brief instruction

OS provides APIs of thread, timer, mutex, semaphore, event, queue, message, memory, interrupt, time and debug.

Header file	Location
aic_osa.h	os\osa\include

2.1 thread

2.1.1 aos_create_thread

Prototype	aos_thread_id aos_create_thread(char *thread_name, aos_entry_func entry_func, void *input, u32 stack_size, u32 queue_num, char *queue_name, u32 priority, u32 autostart)
Function	Create thread.
Constraint	N/A
Parameters	@thread_name: specified thread name. @entry_func: specified thread entry func. @input: specified input param for thread entry func. @stack_size: specified stack size. @queue_num: specified external queue capacity, default message size is sizeof(aos_msg_ptr). @queue_name: specified external queue name. @priority: specified thread priority. @auto_start: specified thread automatic start selection.
Return value	thread id.

2.1.2 aos_create_thread_simply

AICXTEK Confidential

Prototype	#define aos_create_thread_simply(name, func, input, stack_size, priority) \ aos_create_thread(name, func, input, stack_size, 0, AOS_NULL, priority, AOS_AUTO_START)
Function	Simple API to create thread.
Constraint	N/A
Parameters	@name: specified thread name. @func: specified thread entry func. @input: specified input param for thread entry func. @stack_size: specified stack size. @priority: specified thread priority.
Return value	thread id.

2.1.3 aos_delete_thread

Prototype	void aos_delete_thread(aos_thread_id thread_id)
Function	Delete thread.
Constraint	N/A
Parameters	@thread_id: specified thread id.
Return value	N/A

2.1.4 aos_change_thread_pri

Prototype	void aos_change_thread_pri(aos_thread_id thread_id, u32 new_priority, u32 *old_priority)
Function	Change thread priority.
Constraint	N/A
Parameters	@thread_id: specified thread id. @new_priority: new priority to set. @old_priority: return old priority.
Return value	N/A

2.1.5 aos_identify_thread

Prototype	aos_thread_id aos_identify_thread(void)
Function	Get current thread id.
Constraint	N/A
Parameters	N/A
Return value	thread id.

AICXTEK Confidential

2.1.6 aos_suspend_thread

Prototype	aos_status aos_suspend_thread(aos_thread_id thread_id)
Function	Suspend specific thread.
Constraint	N/A
Parameters	@thread_id: specified thread id
Return value	AOS_SUCCESS means success, AOS_ERROR means error.

2.1.7 aos_resume_thread

Prototype	aos_status aos_resume_thread(aos_thread_id thread_id)
Function	Resume specific thread.
Constraint	N/A
Parameters	@thread_id: specified thread id.
Return value	AOS_SUCCESS means success, AOS_ERROR means error.

2.1.8 aos_terminate_thread

Prototype	void aos_terminate_thread(aos_thread_id thread_id)
Function	Terminate specific thread.
Constraint	N/A
Parameters	@thread_id: specified thread id.
Return value	N/A

2.1.9 aos_is_thread_created

Prototype	boolean aos_is_thread_created(aos_thread_id thread_id)
Function	Judge specific thread is created or not.
Constraint	N/A
Parameters	@thread_id: specified thread id.
Return value	N/A

2.2 timer

2.2.1 aos_create_timer

Prototype	aos_timer_ptr aos_create_timer(char *name_ptr, aos_expiration_func expiration_func,

AICXTEK Confidential

	void *input, u32 expiration_time, u32 auto_activate, boolean b_period)
Function	Create specified timer.
Constraint	N/A
Parameters	@name_ptr: specified timer name. @expiration_func: specified timer expiration function. @input: expiration function input parameter. @expiration_time: expiration time (ms). @auto_activate: AOS_AUTO_ACTIVATE or AOS_NO_ACTIVATE. @b_period: AOS_BE_PERIOD or AOS_NO_PERIOD.
Return value	Valid timer pointer means success, AOS_NULL means error.

2.2.2 aos_delete_timer

Prototype	void aos_delete_timer(aos_timer_ptr timer_ptr)
Function	Delete specified timer.
Constraint	N/A
Parameters	@timer_ptr: specified timer ptr.
Return value	N/A

2.2.3 aos_activate_timer

Prototype	void aos_activate_timer(aos_timer_ptr timer_ptr)
Function	Activate specified timer.
Constraint	N/A
Parameters	@timer_ptr: specified timer ptr.
Return value	N/A

2.2.4 aos_deactivate_timer

Prototype	void aos_deactivate_timer(aos_timer_ptr timer_ptr)
Function	Deactivate specified timer.
Constraint	N/A
Parameters	@timer_ptr: specified timer pointer.
Return value	N/A

AICXTEK Confidential

2.2.5 aos_change_timer

Prototype	void aos_change_timer(aos_timer_ptr timer_ptr, u32 expiration_time, boolean b_period)
Function	Change specified timer's expiration time.
Constraint	N/A
Parameters	@timer_ptr: specified timer ptr. @expiration_time: expiration time (ms). @b_period: AOS_BE_PERIOD or AOS_NO_PERIOD.
Return value	N/A

2.2.6 aos_get_remaining_time

Prototype	void aos_get_remaining_time(aos_timer_ptr timer_ptr, u32 *value_ptr)
Function	Get remaining time of specified timer.
Constraint	N/A
Parameters	@timer_ptr: specified timer pointer. @value_ptr: value pointer to save remaining time.
Return value	N/A

2.2.7 aos_is_timer_created

Prototype	boolean aos_is_timer_created(aos_timer_ptr timer_ptr)
Function	Judge specified timer is created or not.
Constraint	N/A
Parameters	@timer_ptr: specified timer pointer.
Return value	AOS_TRUE means timer created, AOS_FALSE means not created.

2.2.8 aos_get_timer_info

Prototype	aos_status aos_get_timer_info(aos_timer_ptr timer_ptr, char **name, u32 *active, u32 *remaining, u32 *reschedule);
Function	Get timer info of specified timer.
Constraint	N/A

AICXTEK Confidential

Parameters	@timer_ptr: specified timer pointer. @name: to save the timer name. @active: to save timer active status. @remaining: to save current remaining time(ms). @reschedule: to save reschedule time(ms) fo period timer.
Return value	AOS_SUCCESS means success , others means error.

2.3 mutex

2.3.1 aos_create_mutex

Prototype	aos_mutex_ptr aos_create_mutex(char *name_ptr)
Function	Create specified mutex.
Constraint	N/A
Parameters	@name_ptr: specified mutex name.
Return value	Valid mutex ptr means success, AOS_NULL means error.

2.3.2 aos_delete_mutex

Prototype	void aos_delete_mutex(aos_mutex_ptr mutex_ptr);
Function	Delete specified mutex.
Constraint	N/A
Parameters	@mutex_ptr: specified mutex ptr.
Return value	N/A

2.3.3 aos_get_mutex

Prototype	aos_status aos_get_mutex(aos_mutex_ptr mutex_ptr, u32 wait_option)
Function	Get specified mutex.
Constraint	N/A
Parameters	@mutex_ptr: specified mutex ptr. @wait_option: AOS_NO_WAIT or AOS_WAIT_FOREVER or actual wait time (ms).
Return value	AOS_SUCCESS means success, other means error.

AICXTEK Confidential

2.3.4 aos_put_mutex

Prototype	void aos_put_mutex(aos_mutex_ptr mutex_ptr)
Function	Put specified mutex.
Constraint	N/A
Parameters	@mutex_ptr: specified mutex ptr.
Return value	N/A

2.4 semaphore

2.4.1 aos_create_semaphore

Prototype	aos_semaphore_ptr aos_create_semaphore(char *name_ptr, u32 initial_count)
Function	Create specified semaphore.
Constraint	N/A
Parameters	@name_ptr: specified semaphore name. @initial_count: initial semaphore count.
Return value	Valid semaphore pointer means success, AOS_NULL means error.

2.4.2 aos_delete_semaphore

Prototype	void aos_delete_semaphore(aos_semaphore_ptr semaphore_ptr)
Function	Delete specified semaphore.
Constraint	N/A
Parameters	@semaphore_ptr: specified semaphore pointer.
Return value	N/A

2.4.3 aos_get_semaphore

Prototype	aos_status aos_get_semaphore(aos_semaphore_ptr semaphore_ptr, u32 wait_option)
Function	Get specified semaphore.
Constraint	N/A
Parameters	@semaphore_ptr: specified semaphore pointer. @wait_option: AOS_NO_WAIT or AOS_WAIT_FOREVER or actual wait time (ms).
Return value	AOS_SUCCESS means success, other means

AICXTEK Confidential

	error.

2.4.4 aos_put_ semaphore

Prototype	void aos_put_semaphore(aos_semaphore_ptr semaphore_ptr)
Function	Put specified semaphore.
Constraint	N/A
Parameters	@semaphore_ptr: specified semaphore pointer.
Return value	N/A

2.5 event

2.5.1 aos_create_event

Prototype	aos_event_ptr aos_create_event(char *name_ptr)
Function	Create specified event.
Constraint	N/A
Parameters	@name_ptr: specified event name.
Return value	Valid event pointer means success, AOS_NULL means error.

2.5.2 aos_delete_event

Prototype	void aos_delete_event(aos_event_ptr event_ptr)
Function	Delete specified event.
Constraint	N/A
Parameters	@event_ptr: specified event pointer.
Return value	N/A

2.5.3 aos_get_event

Prototype	aos_status aos_get_event(aos_event_ptr event_ptr, u32 requested_flags, u32 get_option, u32 *actual_flags_ptr, u32 wait_option)
Function	Get specified event.
Constraint	N/A
Parameters	@event_ptr: specified event ptr. @requested_flags: requested flags.

AICXTEK Confidential

	@get_option: AOS_OR or AOS_OR_CLEAR or AOS_AND or AOS_AND_CLEAR. @actual_flags_ptr: actual flags of event flag group. @wait_option: AOS_NO_WAIT or AOS_WAIT_FOREVER or actual wait time (ms).
Return value	AOS_SUCCESS means success, other means error.

2.5.4 aos_set_event

Prototype	void aos_set_event(aos_event_ptr event_ptr, u32 flags_to_set, u32 set_option)
Function	Set specified event.
Constraint	N/A
Parameters	@event_ptr: specified event ptr. @flags_to_set: flags to set. @set_option: AOS_OR or AOS_AND.
Return value	N/A

2.6 queue

2.6.1 aos_create_queue

Prototype	aos_queue_ptr aos_create_queue(char *queue_name, u32 item_size, u32 queue_num)
Function	Create specified queue.
Constraint	N/A
Parameters	@queue_name: specified queue name. @item_size: specified item size. @queue_num: specified queue capacity.
Return value	Valid queue pointer means success, AOS_NULL means error.

2.6.2 aos_delete_queue

Prototype	void aos_delete_queue(aos_queue_ptr queue_ptr);
Function	Delete specified queue.
Constraint	N/A

AICXTEK Confidential

Parameters	@queue_ptr: specified queue pointer.
Return value	N/A

2.6.3 aos_recv_from_queue

Prototype	aos_status aos_recv_from_queue(aos_queue_ptr queue_ptr, void *item_ptr, u32 wait_option)
Function	Get item from specified queue.
Constraint	N/A
Parameters	@queue_ptr: specified queue pointer. @item_ptr: specified item pointer to store content from queue. @wait_option: AOS_NO_WAIT or AOS_WAIT_FOREVER or actual wait time (ms).
Return value	AOS_SUCCESS means success, others means error.

2.6.4 aos_send_to_queue

Prototype	aos_status aos_send_to_queue(aos_queue_ptr queue_ptr, void *item_ptr, u32 wait_option)
Function	Send specified item to queue tail.
Constraint	N/A
Parameters	@queue_ptr: specified queue pointer. @item_ptr: specified item pointer which points to the content to send. @wait_option: AOS_NO_WAIT or AOS_WAIT_FOREVER or actual wait time (ms).
Return value	AOS_SUCCESS means success, others means error.

2.6.5 aos_send_to_queue_head

Prototype	aos_status aos_send_to_queue_head(aos_queue_ptr queue_ptr, void *item_ptr, u32 wait_option)
Function	Send specified item to queue head.
Constraint	N/A
Parameters	@queue_ptr: specified queue pointer.

AICXTEK Confidential

	@item_ptr: specified item pointer which points to the content to be sent. @wait_option: AOS_NO_WAIT or AOS_WAIT_FOREVER or actual wait time (ms).
Return value	AOS_SUCCESS means success, others means error.

2.7 queue

2.7.1 aos_send_msg

Prototype	u32 aos_send_msg(aos_thread_id dest_id, aos_msg_ptr src_msg_ptr, u32 wait_option)
Function	Send specific message to specific thread queue tail.
Constraint	N/A
Parameters	@dest_id: specified thread id. @src_msg_ptr: specified message pointer to send. @wait_option: AOS_NO_WAIT or AOS_WAIT_FOREVER or actual wait time (ms).
Return value	AOS_SUCCESS means success, AOS_ERROR means error.

2.7.2 aos_send_msg_head

Prototype	u32 aos_send_msg_head(aos_thread_id dest_id, aos_msg_ptr src_msg_ptr, u32 wait_option)
Function	Send specific message to specific thread queue head.
Constraint	N/A
Parameters	@dest_id: specified thread id. @src_msg_ptr: specified message pointer to send. @wait_option: AOS_NO_WAIT or AOS_WAIT_FOREVER or actual wait time (ms).
Return value	AOS_SUCCESS means success, AOS_ERROR means error.

2.7.3 aos_recv_msg

AICXTEK Confidential

Prototype	aos_msg_ptr aos_recv_msg(aos_thread_id dest_id, u32 wait_option)
Function	Receive message from specific thread default queue. Default queue is external queue.
Constraint	N/A
Parameters	@dest_id: specified thread id. @wait_option: AOS_NO_WAIT or AOS_WAIT_FOREVER or actual wait time (ms).
Return value	AOS_NULL or valid msg pointer from queue.

2.8 memory

2.8.1 aos_malloc

Prototype	#define aos_malloc(size)\ aos_malloc_debug((u32)(size), AOS_FALSE, __FILE__, __LINE__)
Function	Allocate memory from block pool or byte pool.
Constraint	N/A
Parameters	@size: memory size to alloc.
Return value	Valid memory pointer means success, AOS_NULL means fail.

2.8.2 aos_zalloc

Prototype	#define aos_zalloc(size)\ aos_malloc_debug((u32)(size), AOS_TRUE, __FILE__, __LINE__)
Function	Allocate memory from block pool or byte pool, and initialize to 0.
Constraint	N/A
Parameters	@size: memory size to alloc.
Return value	Valid memory pointer means success, AOS_NULL means fail.

2.8.3 aos_dma_malloc

Prototype	#define aos_dma_malloc(size)\ aos_dma_malloc_debug((u32)(size), __FILE__, __LINE__)
Function	Allocate memory from block pool or byte pool, and make sure the memory is cacheline-aligned

AICXTEK Confidential

Constraint	N/A
Parameters	@size: memory size to alloc.
Return value	Valid memory pointer means success, AOS_NULL means fail.

2.8.4 aos_free

Prototype	#define aos_free(mem_ptr)\ aos_free_debug((u8 *)(mem_ptr), __FILE__, __LINE__)
Function	Free memory which allocated from heap.
Constraint	N/A
Parameters	@mem_ptr: memory to free.
Return value	AOS_ERROR means error, AOS_SUCCESS means success.

2.8.5 aos_dma_free

Prototype	#define aos_dma_free(mem_ptr)\ aos_dma_free_debug((u8 *)(mem_ptr), __FILE__, __LINE__)
Function	Free dma memory allocated from heap.
Constraint	N/A
Parameters	@mem_ptr: memory to free.
Return value	AOS_ERROR means error, AOS_SUCCESS means success.

2.8.6 aos_is_heap_ptr

Prototype	boolean aos_is_heap_ptr(u8 *dest_ptr)
Function	Check specified memory located in heap or not.
Constraint	N/A
Parameters	@dest_ptr: specified memory pointer to check.
Return value	AOS_TRUE means heap space, AOS_FALSE means other space.

2.9 Interrupt

The process of interrupt are divided to 2 parts: top half and bottom half. The top half process (say interrupt service handler) is processed with interrupt disabled, so it should run as quickly as possible. The bottom half is processed in threads: 2 threads are created, the “hworker” thread which priority is 0, and the “lworker” thread which priority is 1.

AICXTEK Confidential

2.9.1 AOS_INTERRUPT_DECLARE

Prototype	#define AOS_INTERRUPT_DECLARE u32 interrupt_save
Function	Declare a global variable to save irq state.
Constraint	N/A
Parameters	N/A
Return value	N/A

2.9.2 AOS_INTERRUPT_DISABLE

Prototype	#define AOS_INTERRUPT_DISABLE interrupt_save = aos_disable_irq();
Function	Disable the irq of system(set CPSID i and f flag) and save the old irq state of system.
Constraint	N/A
Parameters	N/A
Return value	N/A

2.9.3 AOS_INTERRUPT_RESTORE

Prototype	#define AOS_INTERRUPT_RESTORE aos_restore_irq(interrupt_save)
Function	Restore the irq state of system.
Constraint	N/A
Parameters	N/A
Return value	N/A

2.9.4 aos_request_irq

Prototype	void aos_request_irq(int irq, int (*handler)(void *p), void *p)
Function	Request an irq with isr handler.
Constraint	N/A
Parameters	@irq: the irq number. @handler: the irq handler. @p: the handler callback paramter.
Return value	N/A

2.9.5 aos_request_irq_with_half_bottom

Prototype	void aos_request_irq_with_half_bottom(int irq,

AICXTEK Confidential

	int (*handler)(void *p), void (*half)(void *p), void *p)
Function	Request an irq with isr handler and bottom half handler
Constraint	N/A
Parameters	@irq: the irq number. @handler: the irq handler. @half: the bottom half handler. @p: the handler callback parameter.
Return value	N/A

2.9.6 aos_release_irq

Prototype	void aos_release_irq(int irq)
Function	Release an irq.
Constraint	N/A
Parameters	@irq: the irq number.
Return value	N/A

The interrupt can be disabled as below:

```
#include <aic_osa.h>
void example(void)
{
    AOS_INTERRUPT_DECLARE
    AOS_INTERRUPT_DISABLE
    /* Do something */
    AOS_INTERRUPT_RESTORE
}
```

2.10 time

2.10.1 aos_get_tickcount

Prototype	u32 aos_get_tickcount(void)
Function	Get system tickcount since system bootup.
Constraint	N/A
Parameters	N/A
Return value	Current tickcout (ms).

2.10.2 aos_get_systime

Prototype	aos_systime aos_get_systime(void)
Function	Get system time.
Constraint	N/A
Parameters	N/A
Return value	Current systime.

AICXTEK Confidential

2.10.3 aos_set_systime

Prototype	u32 aos_set_systime(aos_systime *systime_ptr)
Function	Set system time.
Constraint	N/A
Parameters	@systime_ptr: systime to set.
Return value	AOS_SUCCESS means success, AOS_ERROR means error.

2.10.4 aos_time_to_tick

Prototype	u32 aos_time_to_tick(u32 time)
Function	Translate time(ms) to tick.
Constraint	N/A
Parameters	@time: time value(ms) to translate.
Return value	Tick value.

2.10.5 aos_tick_to_time

Prototype	u32 aos_tick_to_time(u32 tick)
Function	Translate tick to time(ms).
Constraint	N/A
Parameters	@tick: tick to translate.
Return value	Time value(ms) .

2.10.6 aos_sleep

Prototype	aos_status aos_sleep(u32 time_ms)
Function	Force specific thread to sleep for time_ms
Constraint	N/A
Parameters	@time_ms: time_ms to sleep.
Return value	AOS_SUCCESS means success, AOS_ERROR means error.

2.11 debug

2.11.1 AOS_ASSERT

Prototype	AOS_ASSERT(n)
Function	Trigger assert if n is FALSE
Constraint	N/A
Parameters	@n: input variable or expression.
Return value	N/A

AICXTEK Confidential

2.11.2 AOS_ASSERT_INFO

Prototype	AOS_ASSERT_INFO(n, fmt, ...)
Function	Trigger assert if n is FALSE, and print assert information
Constraint	N/A
Parameters	@n: input variable or expression. @fmt: information to print.
Return value	N/A

3 Auto init

Brief instruction

The init module provides 7-stage automatic initialization registration interface, In order of initialization is below:
AIC_EARLY_INIT_EXPORT(func)
AIC_DRV_P1_INIT_EXPORT(func)
AIC_DRV_P2_INIT_EXPORT(func)
AIC_COMP_P1_INIT_EXPORT(func)
AIC_COMP_P2_INIT_EXPORT(func)
AIC_APP_P1_INIT_EXPORT(func)
AIC_APP_P2_INIT_EXPORT(func)

Header file	Location
aic_init.h	components\include

3.1 AIC_XXX_INIT_EXPORT

Prototype	AIC_XXX_INIT_EXPORT(fumc)
Function	Register an automatic initialization function for the XXX phase. Can add the macor next to where the function definition ends.
Constraint	N/A
Parameters	@func: the init function name.
Return value	N/A

4 ALOG

Brief instruction

The ALOG module provides 4 levels of log output, The priority from high to low is alog_error, alog_warn, alog_info and alog_debug. By default, alog_debug does not compile. The module you use needs to add its own module header file aic_module_xxx.h. And add the moudule name in the

AICXTEK Confidential

file aic_module.c, than include aic_module_xxx.h.

Header file	Location
alog_api.h	components\include
alog_module_xxx.h	components\include\module

4.1 alog_xxx

Prototype	void alog_xxx(...)
Function	Provide log output at level xxx, and automatically add the module name in the log.
Constraint	N/A
Parameters	Variable arguments to a generic format string.
Return value	N/A

5 DIAG

Brief instruction

The diag modules provide a set of APIs for processing and responding to DIAG commands.
The diag cmd format is as below:
Magic: 4 byte - 0x7e5a5a7e
Sn: 4 byte - the sn of the diag packet.
Sys : 1byte - which sys is the cmd belong to.
Errno: 1byte - the error number of the cmd response. 0 - succ .
Cmd: 1byte - the main cmd that uniformly defined by the diag system.
Subcmd: 2byte- the subcmd that defined by the own module.
Data len: 2byte- the size of the data. If no data, it is 0.
Data： the data_len bytes of data.

Header file	Location
diag_api.h	components\include

5.1 diag_register_cmd_handler

Prototype	void diag_register_cmd_handler(u8 cmd, DIAG_HANDLER_T handler)
Function	Register the cmd handler for the specified cmd.
Constraint	N/A
Parameters	@cmd: the main cmd. @handler: the cmd handler for the specified cmd.

AICXTEK Confidential

Return value	N/A

5.2 diag_cmd_response

Prototype	void diag_cmd_response(diag_rsp_cmt_t *rsp_cmd)
Function	Send a response cmd to pc.
Constraint	N/A
Parameters	@rsp cmd: the responce cmd pointer, can init it by the macro diag_set_rsp.
Return value	N/A

AICXTEK Confidential

6 Network

6.1 HTTP and HTTPS client common api

Briefly instruction

Api for both http and https.

Header file	Location
aic_http_comm.h	aicos\components\network\commom\inc

6.1.1 aic_process_https_url

Prototype	u32 aic_process_https_url (u8 *url, AIC_HTTPS_LOFIN_INFO *login_info, u8 type)
Function	Process URL to get host ip and resource.
Constraint	N/A
Parameters	@url: HTTP(S) URL.

	@login_info: Store the resolved IP address and other information @type: Specify the url type to be processed.
Return value	0x00 for success, others for fail.

6.1.2 aic_https_store_redir_info

Prototype	void aic_https_store_redir_info (u32 handle, u8 *username, u8 *password, u8 *resource, u32 port, u32 total_size, slong start, slong end, u8 *content_type, ulong wait_option)
Function	Store HTTP(S) login information for redirection.
Constraint	N/A
Parameters	@handle: HTTP(S) client handle. @username: username.(Can be NULL if not required by the server). @password:password.(Can be NULL if not required by the server). @sport: HTTP(S) server port.(If it set to 0, the default port(80/443) is used). @total_sizes: total number of bytes will be sent by PUT/POST. @start: start point. @end: end point. @wait_option: suspension option
Return value	void

6.1.3 aic_https_client_set_cid

Prototype	u32 aic_https_client_set_cid (u32 handle, u8 cid)
Function	Set uid used by HTTP(S) client
Constraint	N/A
Parameters	@handle: HTTP(S) client handle. @cid: Context ID
Return value	0x00 for success, others for fail.

6.1.4 aic_https_client_content_type_header_add

Prototype	u32 aic_https_client_content_type_header_add (NX_WEB_HTTP_CLIENT *client_ptr, u8 *resource, u8 *content_type, ulong

AICXTEK Confidential

	wait_option)
Function	Add a custom header to an HTTP(S) request that was previously initilizaed.
Constraint	N/A
Parameters	@client_ptr: HTTP(S) client ptr. @ resource: Server resource name. @ content_type: Destination HTTP(S) type @ wait_option: suspension option
Return value	0x00 for success, others for fail.

6.1.5 aic_https_client_response_body_get

Prototype	u32 aic_https_client_response_body_get(u32 handle, NX_PACKET **packet_ptr, ulong wait_option)
Function	Get a data packet associated with the resource specified by the HTTP(S) GET method. It can be called repeatly to get all of the packets for large resources.
Constraint	N/A
Parameters	@handle: HTTP(S) client handle. @packet_ptr: destination for packet. @wait_option: suspension option.
Return value	0x00 for success, others for fail.

6.1.6 aic_https_client_response_data_get

Prototype	u32 aic_https_client_request_packet_allocate(u32 handle, u8 *buf, u32 buf_bytes, ulong wait_option)
Function	Read specified length of data from the HTTP(S) response.
Constraint	N/A
Parameters	@handle: HTTP(S) client handle. @buf: pointer to destination data area. @buf_bytes: bytes want to read. @wait_option: suspension option.
Return value	Number of bytes received(If success). Less than 0 (If failure).

6.1.7 aic_https_client_request_packet_allocate

Prototype	u32 aic_https_client_request_packet_allocate(u32 handle, NX_PACKET **packet_ptr, ulong

AICXTEK Confidential

	wait_option)
Function	Allocate a packet for HTTP(S) Client requests.
Constraint	The appropriate headers must have been sent in the initial request.
Parameters	@handle: HTTP(S) client handle. @packet_ptr: pointer to allocated packet. @wait_option: suspension option.
Return value	0x00 for success, others for fail.

6.1.8 aic_https_client_put_packet

Prototype	u32 aic_https_client_put_packet(u32 handle, NX_PACKET *packet_ptr, ulong wait_option)
Function	Process a packet of resource data associated with the previous HTTP(S) PUT request.
Constraint	N/A
Parameters	@handle: HTTP(S) client handle. @packet_ptr: resource data packet pointer. @wait_option: suspension option.
Return value	0x00 for success, others for fail.

6.1.9 aic_https_client_put_data

Prototype	u32 aic_https_client_put_packet(u32 handle, u8 *data, ulong data_len, ulong wait_option)
Function	Send the specified length of data to the server.
Constraint	Must be invoked after HTTP(S) POST/PUT request.
Parameters	@handle: HTTP(S) client handle. @data: pointer to data to be send. @data_len: length of data to be send. @wait_option: suspension option.
Return value	0x00 for success, others for fail.

6.1.10 aic_https_client_request_chunked_set

Prototype	u32 aic_https_client_request_chunked_set(u32 handle, u32 chunk_size, NX_PACKET *packet_ptr)
Function	Set the chunked information of chunked request.
Constraint	N/A
Parameters	@handle: HTTP(S) client handle.

AICXTEK Confidential

	@chunk_size: size of the chunk. @packet_ptr: pointer to the packet.
Return value	0x00 for success, others for fail.

6.1.11 aic_https_client_request_header_add

Prototype	u32 aic_https_client_request_header_add(u32 handle, u8 *field_name, u32 name_length, u8 *field_value, u32 value_length, ulong wait_option)
Function	Add a custom header to an HTTP(S) request that was previously initialized with aic_https_client_request_initialization.
Constraint	N/A
Parameters	@handle: HTTP(S) client handle. @field_name: HTTP(S) header field name. @name_length: length of field name string. @field_value: value of header field. @value_length: length of value string. @wait_option: suspension option.
Return value	0x00 for success, others for fail.

6.1.12 aic_https_client_request_initialize

Prototype	u32 aic_https_client_request_initialize(u32 handle, u32 method, u32 input_size, u32 encoding_chunked, u8 *username, u8 *password, ulong wait_option)
Function	Initialize HTTP(S) request using information supplied.
Constraint	N/A
Parameters	@handle: HTTP(S) client handle. @method: HTTP(S) request type (e.g. GET). @input_size: size of input for PUT/POST. @encoding_chunked: chunked encoding flag. If true, input_size is ignored. @username: username.(Can be NULL if not required by the server). @password: password.(Can be NULL if not required by the server). @wait_option: suspension option.
Return value	0x00 for success, others for fail.

6.1.13 aic_https_client_request_send

AICXTEK Confidential

Prototype	u32 aic_https_client_request_send(u32 handle, ulong wait_option)
Function	Send HTTP(S) request to a remote server.
Constraint	The request being sent must have been initialized previously with a call to aic_https_client_request_initialize.
Parameters	@handle: HTTP(S) client handle. @wait_option: suspension option.
Return value	0x00 for success, others for fail.

6.1.14 aic_https_client_response_header_callback_set

Prototype	u32 aic_https_client_response_header_callback_set(u32 handle, void (*callback_function)(char *field_name, u32 field_name_length, char *field_value, u32 field_value_length))
Function	Set callback routine to be invoked when the HTTP(S) Client receives response from the remote server.
Constraint	N/A
Parameters	@handle: HTTP(S) client handle. @callback_function: pointer to callback routine.
Return value	0x00 for success, others for fail.

6.1.15 aic_https_client_total_transfer_bytes_get

Prototype	ulong aic_https_client_total_transfer_bytes_get(u32 handle)
Function	Get the total number of bytes will be sent by the previous POST/PUT start request.
Constraint	N/A
Parameters	@handle: HTTP(S) client handle.
Return value	Total number of bytes.

6.1.16 aic_https_client_actual_bytes_transferred_get

Prototype	ulong aic_https_client_actual_bytes_transferred_get(u32 handle)
Function	Get the actual number of bytes sent by the HTTP(S) POST/PUT.

AICXTEK Confidential

Constraint	N/A
Parameters	@handle: HTTP(S) client handle.
Return value	Actual number of bytes.

6.1.17 aic_https_client_total_receive_bytes_get

Prototype	ulong aic_https_client_total_receive_bytes_get(u32 handle)
Function	Get the total number of bytes will be received from HTTP(S) Server.
Constraint	N/A
Parameters	@handle: HTTP(S) client handle.
Return value	Total number of bytes.

6.1.18 aic_https_client_actual_bytes_received_get

Prototype	ulong aic_https_client_actual_bytes_received_get(u32 handle)
Function	Get the actual number of bytes received from HTTP(S) Server.
Constraint	N/A
Parameters	@handle: HTTP(S) client handle.
Return value	Actual number of bytes.

6.2 Hypertext Transfer Protocol Secure (HTTPS using TLS) Client

Briefly instruction

HTTPS Protocol client API.

Header file	Location
aic_https_client.h	aicos\components\network\commom\inc

6.2.1 aic_https_client_create

Prototype	u32 aic_https_client_create(u8 *client_name)
Function	Create a HTTPS Client on the specified IP.
Constraint	N/A
Parameters	@client_name: HTTPS Client name.
Return value	HTTPS client handle(!0). 0x00 for fail.

AICXTEK Confidential

6.2.2 aic_https_client_delete

Prototype	u32 aic_https_client_delete(u32 handle)
Function	This function deletes a previously created HTTPS Client on the specified IP.
Constraint	N/A
Parameters	@handle: HTTPS client handle.
Return value	0x00 for success, others for fail.

6.2.3 aic_https_ca_crt_add

Prototype	u32 aic_https_ca_crt_add (u32 handle, u8 *crt_data, u16 crt_len, u8 *crt_path, u8 crt_cid)
Function	This function adds a CA Certificate to our trusted store for verifying incoming server certificates.
Constraint	If the certificate path is set, the system must support the file system.Only certificates in der format are supported.
Parameters	@handle: HTTPS client handle. @crt_data: certificate data. @crt_len: certificate data length. @crt_path:the path of certificates stored locally.If there is no local certificate, it can be set to NULL. @crt_id: certificate id.
Return value	0x00 for success, others for fail.

6.2.4 aic_https_ca_crt_remove

Prototype	u32 aic_https_ca_crt_remove(u32 handle, u8 crt_id)
Function	This function removes an NX_SECURE_TLS_CERTIFICATE instance from the trusted certificates store, keyed on the Common Name filed.
Constraint	N/A
Parameters	@handle: HTTPS client handle. @crt_id: certificate id.
Return value	0x00 for success, others for fail.

6.2.5 aic_https_client_crt_add

Prototype	u32 aic_https_client_crt_add (u32 handle, u8

AICXTEK Confidential

	*crt_data, u16 crt_len, const u8 *priv_key, u16 priv_len, u32 key_type, u8 *crt_path, u8 *ker_path, u8 crt_cid)
Function	This function adds a CA Certificate to our local store for verifying incoming server certificates
Constraint	If the certificate path is set, the system must support the file system. Only certificates in der format are supported.
Parameters	@handle: HTTPS client handle. @crt_data: certificate data. @crt_len: certificate data length. @ priv_key: private key data. @priv_len: length of private key. @key_type: type of private key data. @crt_path:the path of certificates stored locally.If there is no local certificate, it can be set to NULL. @key_path: the path of private key stored locally.If there is no local private key, it can be set to NULL. @crt_id: certificate id.
Return value	0x00 for success, others for fail.

6.2.6 aic_https_client_crt_remove

Prototype	u32 aic_https_client_crt_remove(u32 handle, u8 crt_id)
Function	This function removes an NX_SECURE_TLS_CERTIFICATE instance from the local certificates store, keyed on the Common Name filed.
Constraint	N/A
Parameters	@handle: HTTPS client handle. @crt_id: certificate id.
Return value	0x00 for success, others for fail.

6.2.7 aic_https_client_get_start

Prototype	u32 aic_https_client_get_start(u32 handle, u8 *url, u8 *username, u8 *password, u32 server_port, ulong wait_option)
Function	Process the HTTPS GET request.

AICXTEK Confidential

Constraint	N/A
Parameters	@handle: HTTPS client handle. @url: HTTPS url. @username: username.(Can be NULL if not required by the server). @password:password.(Can be NULL if not required by the server). @server_port: HTTPS server port.(If it set to 0, the default port(443) is used). @wait_option: suspension option
Return value	0x00 for success, others for fail.

6.2.8 aic_https_client_range_get_start

Prototype	u32 aic_https_client_range_get_start(u32 handle, u8 *url, u8 *username, u8 *password, u32 server_port, slong start, slong end, ulong wait_option)
Function	Process the HTTPS GET request.
Constraint	N/A
Parameters	@handle: HTTPS client handle. @url: HTTPS url. @username: username.(Can be NULL if not required by the server). @password:password.(Can be NULL if not required by the server). @server_port: HTTPS server port.(If it set to 0, the default port(443) is used). @start: start point. @end: end point. (If end is smaller than start, the range is from start to end of file. If end is greater than start, the range is from start to end. If both start and end are less than 0, get all files. If start is less than 0 and end is greater than 0, get the specified bytes from the end of the file forward.) @wait_option: suspension option
Return value	0x00 for success, others for fail.

AICXTEK Confidential

6.2.9 aic_https_client_post_start

Prototype	u32 aic_https_client_post_start(u32 handle, u8 *url, u8 *username, u8 *password, u32 server_port, ulong total_bytes, ulong wait_option)
Function	Process the HTTPS POST request.
Constraint	N/A
Parameters	@handle: HTTPS client handle. @url: HTTPS url. @username: username.(Can be NULL if not required by the server). @password: password.(Can be NULL if not required by the server). @server_port: HTTPS server port.(If it set to 0, the default port(443) is used). @total_bytes: total number of bytes will be sent by POST. @wait_option: suspension option
Return value	0x00 for success, others for fail.

6.2.10 aic_https_client_post_start_ext

Prototype	u32 aic_https_client_post_start_ext(u32 handle, u8 *url, u8 *username, u8 *password, u32 server_port, ulong total_bytes, u8 *content_type, ulong wait_option)
Function	Process the HTTPS POST request. User can specify content type.
Constraint	N/A
Parameters	@handle: HTTPS client handle. @url: HTTPS url. @username: username.(Can be NULL if not required by the server). @password: password.(Can be NULL if not required by the server). @server_port: HTTPS server port.(If it set to 0, the default port(443) is used). @total_bytes: total number of bytes will be sent by POST. @content_type: content type add to HTTPS POST request packet header. @wait_option: suspension option
Return value	0x00 for success, others for fail.

AICXTEK Confidential

6.2.11 aic_https_client_put_start

Prototype	u32 aic_https_client_put_start(u32 handle, u8 *url, u8 *username, u8 *password, u32 server_port, ulong total_bytes, ulong wait_option)
Function	Process the HTTPS PUT request.
Constraint	N/A
Parameters	@handle: HTTPS client handle. @url: HTTPS url. @username: username.(Can be NULL if not required by the server). @password: password.(Can be NULL if not required by the server). @server_port: HTTPS server port.(If it set to 0, the default port(443) is used). @total_bytes: total number of bytes will be sent by PUT. @wait_option: suspension option
Return value	0x00 for success, others for fail.

6.2.12 aic_https_client_head_start

Prototype	u32 aic_https_client_head_start(u32 handle, u8 *url, u8 *username, u8 *password, u32 server_port, ulong wait_option)
Function	Process the HTTPS HEAD request.
Constraint	N/A
Parameters	@handle: HTTPS client handle. @url: HTTPS url. @username: username.(Can be NULL if not required by the server). @password: password.(Can be NULL if not required by the server). @server_port: HTTPS server port.(If it set to 0, the default port(443) is used). @wait_option: suspension option.
Return value	0x00 for success, others for fail.

6.2.13 aic_https_client_delete_start

Prototype	u32 aic_https_client_delete_start(u32 handle, u8 *url, u8 *username, u8 *password, u32 server_port, ulong wait_option)

AICXTEK Confidential

Function	Process the HTTPS DELETE request.
Constraint	N/A
Parameters	@handle: HTTPS client handle. @url: HTTPS url. @username: username.(Can be NULL if not required by the server). @password: password.(Can be NULL if not required by the server). @server_port: HTTPS server port.(If it set to 0, the default port(443) is used). @wait_option: suspension option.
Return value	0x00 for success, others for fail.

6.2.14 aic_https_client_secure_connect

Prototype	u32 aic_https_client_secure_connect(u32 handle, ulong wait_option)
Function	Connect HTTPS Client on the specified IP.
Constraint	N/A
Parameters	@handle: HTTPS client handle. @wait_option: suspension option.
Return value	0x00 for success, others for fail.

6.3 Hypertext Transfer Protocol Secure (HTTPS using TLS) Server

Briefly instruction

HTTPS Protocol server API.

Header file	Location
aic_https_server.h	aicos\components\network\https

6.3.1 aic_https_server_create

Prototype	u32 aic_https_server_create(NX_WEB_HTTP_SERVER *server_ptr, AIC_HTTPS_SERVER_INFO *server_info, u32 (*authentication_check) (NX_WEB_HTTP_SERVER *server_ptr, u32 request_type, char *resource, char **name, char **password, char **realm), u32 (*request_notify)

AICXTEK Confidential

	(NX_WEB_HTTP_SERVER *server_ptr, u32 request_type, char *resource, NX_PACKET *packet_ptr))
Function	Create HTTPS server on the specified IP.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @server_info: information needed for creating https server. Such as the server port. @authentication_check: pointer to application's authentication checking. @request_notify: pointer to application's request notify service.
Return value	0x00 for success, others for fail.

6.3.2 aic_https_server_delete

Prototype	u32 aic_https_server_delete(NX_WEB_HTTP_SERVER *server_ptr)
Function	Delete previously created HTTPS server on the pecified IP.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server.
Return value	0x00 for success, others for fail.

6.3.3 aic_https_server_cache_info_callback_set

Prototype	u32 aic_https_server_cache_info_callback_set(NX_WEB_HTTP_SERVER *server_ptr, u32 (*cache_info_get)(char *, u32 *, NX_WEB_HTTP_SERVER_DATE *))
Function	Set cache info callback function.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @cache_info_get: pointer to application's cache information function.
Return value	0x00 for success, others for fail.

6.3.4 aic_https_server_callback_data_send

Prototype	u32 aic_https_server_callback_data_send(NX_WEB_HTTP_SERVER *server_ptr, void *data_ptr, ulong data_length)

AICXTEK Confidential

Function	Send data to the client from the application callback function. This is typically done to satisfy a GET or POST request that is processed completely by the application callback function.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @data_ptr: pointer to data to send. @data_length: length of data to send
Return value	0x00 for success, others for fail.

6.3.5 aic_https_server_callback_generate_response_header

Prototype	u32 aic_https_server_callback_generate_response_header(NX_WEB_HTTP_SERVER *server_ptr, NX_PACKET **packet_pptr, AIC_HTTPS_SERVER_RESP *resp)
Function	This function is used in callback function to generate HTTPS response header.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @packet_pptr: pointer to packet. @resp: information to reply.
Return value	0x00 for success, others for fail.

6.3.6 aic_https_server_callback_packet_send

Prototype	u32 aic_https_server_callback_packet_send(NX_WEB_HTTP_SERVER *server_ptr, NX_PACKET *packet_ptr)
Function	This function is used in callback function to send a packet.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @packet_pptr: pointer to packet.
Return value	0x00 for success, others for fail.

6.3.7 aic_https_server_callback_response_send

Prototype	u32 aic_https_server_callback_response_send(NX_WEB_HTTP_SERVER *server_ptr, AIC_HTTPS_SERVER_RESP *resp)
Function	This function sends the completion response

AICXTEK Confidential

	to the client from the application callback function. This is typically done to satisfy a GET or POST request that is processed completely by the application callback function.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @resp: information to reply.
Return value	0x00 for success, others for fail.

6.3.8 aic_https_server_content_get

Prototype	u32 aic_https_server_content_get(NX_WEB_HTTP_SERVER *server_ptr, NX_PACKET *packet_ptr, ulong byte_offset, char *destination_ptr, u32 destination_size, u32 *actual_size)
Function	Return the user's specified portion of the HTTPS content.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @packet_ptr: pointer to HTTPS request packet. @byte_offset: byte offset into content. @destination_ptr: pointer to content destination. @destination_size: maximum size of destination. @actual_size: actual amount of content.
Return value	0x00 for success, others for fail.

6.3.9 aic_https_server_content_length_get

Prototype	u32 aic_https_server_content_length_get(NX_PACKET *packet_ptr, ulong *content_length)
Function	Find valid content length field and return completion status.
Constraint	N/A
Parameters	@packet_ptr: pointer to HTTPS request packet. @content_length: pointer to a valid content length value.

AICXTEK Confidential

Return value	0x00 for success, others for fail.

6.3.10 aic_https_server_get_entity_content

Prototype	u32 aic_https_server_get_entity_content(NX_WEB_HTTP_SERVER *server_ptr, NX_PACKET **packet_pptr, ulong *available_offset, ulong *available_length)
Function	Get the entity content.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @packet_pptr: pointer to packet. @available_offset: return the offset. @available_length: return the length.
Return value	0x00 for success, others for fail.

6.3.11 aic_https_server_get_entity_header

Prototype	u32 aic_https_server_get_entity_header(NX_WEB_HTTP_SERVER *server_ptr, NX_PACKET **packet_pptr, char *entity_header_buffer, ulong buffer_size)
Function	Get the entity header and skip header.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @packet_pptr: pointer to packet. @entity_header_buffer: buffer to get entity header. @buffer_size: size of buffer.
Return value	0x00 for success, others for fail.

6.3.12 aic_https_server_gmt_callback_set

Prototype	u32 aic_https_server_gmt_callback_set(NX_WEB_HTTP_SERVER *server_ptr, void (*gmt_get)(NX_WEB_HTTP_SERVER_DATE *))
Function	Set gmt callback function.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @gmt_get: pointer to application's GMT time function.

AICXTEK Confidential

Return value	0x00 for success, others for fail.

6.3.13 aic_https_server_invalid_userpassword_notify_set

Prototype	u32 aic_https_server_invalid_userpassword_notify_set(NX_WEB_HTTP_SERVER *server_ptr, u32 (*invalid_username_password_callback)(char *resource, NXD_ADDRESS *client_nxd_address, u32 request_type))
Function	Set invalid username password callback function.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @invalid_username_password_callback: pointer to application's invalid username password callback function.
Return value	0x00 for success, others for fail.

6.3.14 aic_https_server_mime_maps_additional_set

Prototype	u32 aic_https_server_mime_maps_additional_set(NX_WEB_HTTP_SERVER *server_ptr, NX_WEB_HTTP_SERVER_MIME_MAP *mime_maps, u32 mime_maps_num)
Function	Set additional MIME maps.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @mime_maps: pointer MIME map array. @mime_maps_num: number of MIME map array.
Return value	0x00 for success, others for fail.

6.3.15 aic_https_server_response_packet_allocate

Prototype	u32 aic_https_server_response_packet_allocate(NX_WEB_HTTP_SERVER *server_ptr, NX_PACKET **packet_ptr, u32 wait_option)
Function	Allocate packet for Server HTTPS response.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server.

AICXTEK Confidential

	@packet_ptr: pointer to allocated packet. @wait_option: suspension option.
Return value	0x00 for success, others for fail.

6.3.16 aic_https_server_packet_content_find

Prototype	u32 aic_https_server_packet_content_find(NX_WEB_HTTP_SERVER *server_ptr, NX_PACKET **packet_ptr, ulong *content_length)
Function	Find the content length specified in the HTTPS header, and move the nx_packet_prepend_ptr to the beginning of the content.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @packet_ptr: pointer to pointer to the packet that contains the HTTPS header @content_length: pointer to content length.
Return value	0x00 for success, others for fail.

6.3.17 aic_https_server_packet_get

Prototype	u32 aic_https_server_packet_get(NX_WEB_HTTP_SERVER *server_ptr, NX_PACKET **packet_ptr)
Function	Obtain the next packet from the HTTPS server socket.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @packet_ptr: pointer to the packet to be returned.
Return value	0x00 for success, others for fail.

6.3.18 aic_https_server_param_get

Prototype	u32 aic_https_server_param_get(NX_PACKET *packet_ptr, u32 param_number, char *param_ptr, u32 *param_size, u32 max_param_size)
Function	Return the requested parameter specified in the URL requested by the client.
Constraint	N/A

AICXTEK Confidential

Parameters	@packet_ptr: pointer to HTTPS request packet. @param_number: parameter number (start at 0). @param_ptr: pointer to destination. @param_size: pointer to total parameter data length return. @max_param_size: maximum size of destination.
Return value	0x00 for success, others for fail.

6.3.19 aic_https_server_query_get

Prototype	u32 aic_https_server_query_get(NX_PACKET *packet_ptr, u32 query_number, char *query_ptr, u32 *query_size, u32 max_query_size)
Function	Return the requested query specified in the URL requested by the client.
Constraint	N/A
Parameters	@packet_ptr: pointer to HTTPS request packet. @query_number: query number (start at 0). @query_ptr: pointer to destination. @query_size: pointer to data size return. @max_query_size: maximum size of destination.
Return value	0x00 for success, others for fail.

6.3.20 aic_https_server_response_chunked_set

Prototype	u32 aic_https_server_response_chunked_set(NX_WEB_HTTP_SERVER *server_ptr, u32 chunk_size, NX_PACKET *packet_ptr)
Function	Set the chunked information of chunked response.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @chunk_size: size of the chunk. @packet_ptr: pointer to the packet.
Return value	0x00 for success, others for fail.

AICXTEK Confidential

6.3.21 aic_https_server_secure_configure

Prototype	u32 aic_https_server_secure_configure(NX_WEB_HTTP_SERVER *server_ptr, const NX_SECURE_TLS_CRYPTO *crypto_table, AIC_HTTPS_SERVER_SECURE_CRT *server_crt, AIC_HTTPS_SERVER_TRUSTED_CRT *trusted_crt, AIC_HTTPS_SERVER_REMOTE_CRT *remote_crt)
Function	Configure previously created HTTPS server instance to use TLS for secure HTTPS communications.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @crypto_table: TLS cryptographic routines. @server_crt: TLS server certificate information. @trusted_crt: TLS trusted certificates information. @remote_crt: remote certificates information.
Return value	0x00 for success, others for fail.

6.3.22 aic_https_server_start

Prototype	u32 aic_https_server_start(NX_WEB_HTTP_SERVER *server_ptr)
Function	Start previously created HTTPS server on the specified IP.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server.
Return value	0x00 for success, others for fail.

6.3.23 aic_https_server_stop

Prototype	u32 aic_https_server_stop(NX_WEB_HTTP_SERVER *server_ptr)
Function	Stop previously created HTTPS server on the specified IP.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server.
Return value	0x00 for success, others for fail.

6.3.24 aic_https_server_type_get

AICXTEK Confidential

Prototype	u32 aic_https_server_type_get(NX_WEB_HTTP_SERVER *server_ptr, AIC_HTTPS_SERVER_TYPE_INFO *type)
Function	Get server type.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @type: HTTPS server type information.
Return value	0x00 for success, others for fail.

6.3.25 aic_https_server_digest_authenticate_notify_set

Prototype	u32 aic_https_server_digest_authenticate_notify_set(NX_WEB_HTTP_SERVER *server_ptr, u32 (*digest_authenticate_callback)(NX_WEB_HTTP_SERVER *server_ptr, char *name_ptr, char *realm_ptr, char *password_ptr, char *method, char *authorization_uri, char *authorization_nc, char *authorization_cnonce))
Function	Set digest authenticate callback function.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @digest_authenticate_callback: pointer to application's digest authenticate callback function.
Return value	0x00 for success, others for fail.

6.3.26 aic_https_server_authentication_check_set

Prototype	u32 aic_https_server_authentication_check_set(NX_WEB_HTTP_SERVER *server_ptr, u32 (*authentication_check_extended)(NX_WEB_HTTP_SERVER *server_ptr, u32 request_type, char *resource, char **name, u32 *name_length, char **password, u32 *password_length, char **realm, u32 *realm_length))
Function	Set the callback function of authentication checking.
Constraint	N/A
Parameters	@server_ptr: pointer to HTTPS server. @authentication_check_extended: pointer to application's authentication checking.
Return value	0x00 for success, others for fail.

6.4 Hypertext Transfer Protocol (HTTP) Client

Briefly instruction

AICXTEK Confidential

HTTP Protocol client API.

Header file	Location
aic_http_client.h	aicos\components\network\commom\inc

6.4.1 aic_http_client_create

Prototype	u32 aic_http_client_create(u8 *client_name)
Function	Create a HTTP Client on the specified IP.
Constraint	N/A
Parameters	@client_name: HTTP Client name.
Return value	HTTP client handle(!0). 0x00 for fail.

6.4.2 aic_http_client_delete

Prototype	u32 aic_http_client_delete(u32 handle)
Function	This function deletes a previously created HTTP Client on the specified IP.
Constraint	N/A
Parameters	@handle: HTTP client handle.
Return value	0x00 for success, others for fail.

6.4.3 aic_http_client_get_start

Prototype	u32 aic_http_client_get_start(u32 handle, u8 *url, u8 *username, u8 *password, u32 server_port, ulong wait_option)
Function	Process the HTTP GET request.
Constraint	N/A
Parameters	@handle: HTTP client handle. @url: HTTP url. @username: username.(Can be NULL if not required by the server). @password:password.(Can be NULL if not required by the server). @server_port: HTTP server port.(If it set to 0, the default port(80) is used). @wait_option: suspension option
Return value	0x00 for success, others for fail.

6.4.4 aic_http_client_range_get_start

Prototype	u32 aic_http_client_range_get_start(u32 handle, u8 *url, u8 *username, u8 *password, u32 server_port, slong start, slong end, ulong

AICXTEK Confidential

	wait_option)
Function	Process the HTTP GET request.
Constraint	N/A
Parameters	@handle: HTTP client handle. @url: HTTP url. @username: username.(Can be NULL if not required by the server). @password:password.(Can be NULL if not required by the server). @server_port: HTTP server port.(If it set to 0, the default port(80) is used). @start: start point. @end: end point. (If end is smaller than start, the range is from start to end of file. If end is greater than start, the range is from start to end. If both start and end are less than 0, get all files. If start is less than 0 and end is greater than 0, get the specified bytes from the end of the file forward.) @wait_option: suspension option
Return value	0x00 for success, others for fail.

6.4.5 aic_http_client_post_start

Prototype	u32 aic_http_client_post_start(u32 handle, u8 *url, u8 *username, u8 *password, u32 server_port, ulong total_bytes, ulong wait_option)
Function	Process the HTTP POST request.
Constraint	N/A
Parameters	@handle: HTTP client handle. @url: HTTP url. @username: username.(Can be NULL if not required by the server). @password: password.(Can be NULL if not required by the server). @server_port: HTTP server port.(If it set to 0, the default port(80) is used). @total_bytes: total number of bytes will be

AICXTEK Confidential

	sent by POST. @wait_option: suspension option
Return value	0x00 for success, others for fail.

6.4.6 aic_http_client_post_start_ext

Prototype	u32 aic_http_client_post_start_ext(u32 handle, u8 *url, u8 *username, u8 *password, u32 server_port, ulong total_bytes, u8 *content_type, ulong wait_option)
Function	Process the HTTP POST request. User can specify content type.
Constraint	N/A
Parameters	@handle: HTTP client handle. @url: HTTP url. @username: username.(Can be NULL if not required by the server). @password: password.(Can be NULL if not required by the server). @server_port: HTTP server port.(If it set to 0, the default port(80) is used). @total_bytes: total number of bytes will be sent by POST. @content_type: content type add to HTTP POST request packet header. @wait_option: suspension option
Return value	0x00 for success, others for fail.

6.4.7 aic_http_client_put_start

Prototype	u32 aic_http_client_put_start(u32 handle, u8 *url, u8 *username, u8 *password, u32 server_port, ulong total_bytes, ulong wait_option)
Function	Process the HTTP PUT request.
Constraint	N/A
Parameters	@handle: HTTP client handle. @url: HTTP url. @username: username.(Can be NULL if not required by the server). @password: password.(Can be NULL if not required by the server). @server_port: HTTP server port.(If it set to 0, the default port(80) is used).

AICXTEK Confidential

	@total_bytes: total number of bytes will be sent by PUT. @wait_option: suspension option
Return value	0x00 for success, others for fail.

6.4.8 aic_http_client_head_start

Prototype	u32 aic_http_client_head_start(u32 handle, u8 *url, u8 *username, u8 *password, u32 server_port, ulong wait_option)
Function	Process the HTTP HEAD request.
Constraint	N/A
Parameters	@handle: HTTP client handle. @url: HTTP url. @username: username.(Can be NULL if not required by the server). @password: password.(Can be NULL if not required by the server). @server_port: HTTP server port.(If it set to 0, the default port(80) is used). @wait_option: suspension option.
Return value	0x00 for success, others for fail.

6.4.9 aic_http_client_delete_start

Prototype	u32 aic_http_client_delete_start(u32 handle, u8 *url, u8 *username, u8 *password, u32 server_port, ulong wait_option)
Function	Process the HTTP DELETE request.
Constraint	N/A
Parameters	@handle: HTTP client handle. @url: HTTP url. @username: username.(Can be NULL if not required by the server). @password: password.(Can be NULL if not required by the server). @server_port: HTTP server port.(If it set to 0, the default port(80) is used). @wait_option: suspension option.
Return value	0x00 for success, others for fail.

6.4.10 aic_http_client_connect

Prototype	u32 aic_http_client_connect(u32 handle,

AICXTEK Confidential

	ulong wait_option)
Function	Connect HTTP Client on the specified IP.
Constraint	N/A
Parameters	@handle: HTTP client handle. @wait_option: suspension option.
Return value	0x00 for success, others for fail.

6.5 BSD Socket

Briefly instruction

BSD Socket API.

Header file	Location
aic_net_api.h	aicos\ components\network\common\inc

6.5.1 socket

Prototype	INT socket(INT protocolFamily, INT type, INT protocol)
Function	Creates a TCP or UDP socket, which may then be used as an end point of communication for sending and receiving data using the specified protocol.
Constraint	
Parameters	@protocolFamily: Protocol family e.g AF_INET @type: Type of the socket TCP or UDP @protocol: Socket protocol
Return value	Return socket descriptor on success; return -1 On failure

6.5.2 connect

Prototype	INT connect(INT sockID, struct sockaddr* remoteAddress, INT addressLength)
Function	Establishes a connection between a client socket and a remote server socket associated with the remote address, if any. Upon returning successfully, the given socket's local and remote IP address and port information are filled in. If the socket was not previously bound to a local port, one is assigned

AICXTEK Confidential

	randomly.
Constraint	
Parameters	@sockID: Socket descriptor. @remoteAddress: Remote address structure. @addressLength: Address structure length.
Return value	Return 0 on success; return -1 on failure.

6.5.3 bind

Prototype	INT bind(INT sockID, struct sockaddr * localAddress, INT addressLength)
Function	This function binds a socket to a local port. The port in the struct in the struct sockaddr structure may be wildcarded, in which case NetX will select a port number.
Constraint	
Parameters	@sockID: Socket descriptor. @localAddress: Populated socket address. @addressLength: Socket address length.
Return value	Return 0 on success; return -1 on failure.

6.5.4 listen

Prototype	INT listen(INT sockID, INT backlog)
Function	This function sets the given socket ready to accept incoming client connections. The socket must already be associated with a local port which means the bind() must have been called previously. After this call, incoming TCP connections requests addressed to the local port (and IP address, if specified previously) will be completed & queued until they are passed to the program via accept().
Constraint	
Parameters	@sockID: socket descriptor. @backlog: Maximum number of new connections queued.
Return value	Return 0 on success; return -1 on failure.

6.5.5 accept

Prototype	INT accept(INT sockID, struct sockaddr * ClientAddress, INT *addressLength)

AICXTEK Confidential

Function	This function blocks while waiting for connections addressed to the IP address and port to which this socket is bound. A listen() must previously have been called on this given socket.
Constraint	
Parameters	@sockID: socket descriptor. @clientAddress: Originating socket IP address. and port. @addressLength: Length of sockaddr buffer (in) returned address (out).
Return value	Return socket ID for new connection if success; return -1 if failure.

6.5.6 send

Prototype	INT send(INT sockID, const CHAR *msg, INT msgLength, INT flags)
Function	This function sends a packet out the given socket. When the call returns, the data has been queued for transmission over the connection. The return value indicates the number of byes actually transmitted.
Constraint	
Parameters	@sockID: Socket. @msg: Data to be transmitted. @msgLength: Number of bytes to be sent. @flags: Control flags, support MSG_DONTWAIT.
Return value	Return number of bytes sent If successful; return -1 If failure.

6.5.7 sendto

Prototype	INT sendto(INT sockID, CHAR *msg, INT msgLength, INT flags, struct sockaddr *destAddr, INT destAddrLen)
Function	This function sends a packet out the given socket. When the call returns, the data has been queued for transmission over the connection. To use sendto on a TCP BSD socket, the socket must already be in connected state.

AICXTEK Confidential

Constraint	
Parameters	@sockID: Socket (must be connected). @msg: Data to transmit. @msgLength: Number of bytes to send. @flags: Control flags, support. MSG_DONTWAIT @sockaddr: Destination address. @destAddrLen: Length of destination address.
Return value	Return number of bytes sent If no error occurs; return NX_SOC_ERROR (-1) In case of socket error.

6.5.8 recv

Prototype	INT recv(INT sockID, VOID *rcvBuffer, INT bufferLength, INT flags)
Function	This function copies up to a specified number of bytes received on the socket into specified location. The given socket must be in the connected state. Normally, the call blocks until either at least one byte is returned or the connection closes.The return value indicates the number of bytes actually copied into the buffer starting at the specified location.
Constraint	
Parameters	@sockID: Socket (must be connected). @rcvBuffer: Pointer to put data received. @bufferLength: Maximum bytes in buffer. @flags: Control flags, support MSG_PEEK and MSG_DONTWAIT.
Return value	Return number of bytes received If success; return -1 If failure; return 0 when socket disconnected.

6.5.9 recvfrom

Prototype	INT recvfrom(INT sockID, CHAR *rcvBuffer, INT bufferLength, INT flags, struct sockaddr *fromAddr, INT *fromAddrLen)
Function	This function copies up to a specified number of bytes, received on the socket into a specified location. To use recvfrom() on a TCP socket requires the socket to be in the

AICXTEK Confidential

	connected state.
Constraint	
Parameters	@sockID: Socket(must be connected). @buffer: Pointer to hold data received. @bufferSize: Maximum number of bytes. @flags: Control flags, support MSG_PEEK and MSG_DONTWAIT. @fromAddr: Address data of sender. @fromAddrLen: Length of address structure.
Return value	Return number of bytes received If no error occurs; return -1 In case of any error.

6.5.10 soc_close

Prototype	INT soc_close(INT sockID)
Function	This function terminates communications on the supplied BSD socket. The socket is disallowed for further sends and receives. Socket resources such as socket memory are returned to the system.
Constraint	
Parameters	@sockID: Socket ID.
Return value	Return 0 On success; return -1 On failure.

6.5.11 getaddrinfo

Prototype	INT getaddrinfo(const CHAR *node, const CHAR *service, const struct addrinfo *hints, struct addrinfo **res)
Function	This function is used for DNS parsing.
Constraint	N/A
Parameters	@node: node is either a hostname or an address string. @service: either a service name or a decimal port number string. @hints: either a null pointer or a pointer to an addrinfo structure that the caller fills in with hints about the types of information the caller wants returned @res: Pointer to the returned addrinfo list.
Return value	Return 0 On success; return other On failure.

6.5.12 aic_socket_register_establish_notify

AICXTEK Confidential

Prototype	int aic_socket_register_establish_notify(int sockID, void (*tcp_establish_notify)(int sock_id, void *pdata), void *pdata)
Function	This function is establishing a connection callback to registers with BSD socket.
Constraint	N/A
Parameters	@sockID: Socket ID. @tcp_establish_notify: Establish Callback API. @pdata: Extended parameters.
Return value	Return 0 On success; return other On failure.

6.5.13 aic_socket_unregister_establish_notify

Prototype	int aic_socket_unregister_establish_notify(INT sockID)
Function	This function unregisters the callback for establishing a connection.
Constraint	N/A
Parameters	@sockID: Socket ID.
Return value	Return 0 On success; return other On failure.

6.5.14 aic_tcp_socket_retransmit_configure

Prototype	int aic_tcp_socket_retransmit_configure(int sockID, ULONG max_retries, ULONG max_timeout))
Function	This function is used to configure TCP retransmission parameters.
Constraint	N/A
Parameters	@sockID: Socket ID. @max_retries: Maximum of restransmissions count @max_timeout:Maximum restransmission time
Return value	Return 0 On success; return other On failure.

6.5.15 aic_tcp_socket_transmit_max_queue_configure

Prototype	int aic_tcp_socket_transmit_max_queue_configure(int sockID, u32 max_queue_depth)
Function	This function is used to configure the number of TCP retransmission packet cache.
Constraint	N/A

AICXTEK Confidential

Parameters	@sockID: Socket ID. @max_queue_depth: Retransmit maximum cache queue.
Return value	Return 0 On success; return other On failure.

6.5.16 aic_socket_register_connect_notify

Prototype	int aic_socket_register_connect_notify(int sockID, void (*tcp_connect_notify)(int sock_id, u32 status, void *pdata), void *pdata)
Function	This function is connect callback to registers with BSD socket.
Constraint	N/A
Parameters	@sockID: Socket ID. @tcp_connect_notify: Connect Callback API. 1) sock_id: Socket ID. 2) status: connect status. 3) pdata: Extended parameters. @pdata: Extended parameters.
Return value	Return 0 On success; return other On failure.

6.5.17 aic_socket_unregister_connect_notify

Prototype	int aic_socket_unregister_connect_notify(int sockID)
Function	This function unregisters the callback for connect.
Constraint	N/A
Parameters	@sockID: Socket ID.
Return value	Return 0 On success; return other On failure.

6.5.18 aic_socket_register_receive_notify

Prototype	int aic_socket_register_receive_notify(int sockID, void (*tcp_recv_notify)(INT sock_id, void *pdata), void *pdata)
Function	This function is receive callback to registers with BSD socket.
Constraint	N/A
Parameters	@sockID: Socket ID. @tcp_recv_notify: Receive Callback API. @pdata: Extended parameters.
Return value	Return 0 On success; return other On failure.

AICXTEK Confidential

6.5.19 aic_socket_unregister_receive_notify

Prototype	int aic_socket_unregister_receive_notify(int sockID)
Function	This function unregisters the callback for receive.
Constraint	N/A
Parameters	@sockID: Socket ID.
Return value	Return 0 On success; return other On failure.

6.5.20 aic_socket_register_disconnect_notify

Prototype	int aic_socket_register_tcp_disconnect_notify(int sockID, void (*tcp_disconnect_notify)(int sock_id, void *pdata, int is_active), void *pdata)
Function	This function is disconnect callback to registers with BSD socket.
Constraint	N/A
Parameters	@sockID: Socket ID. @tcp_disconnect_notify: Disconnect Callback API. @pdata: Extended parameters.
Return value	Return 0 On success; return other On failure.

6.5.21 aic_socket_unregister_disconnect_notify

Prototype	int aic_socket_unregister_disconnect_notify(int sockID)
Function	This function unregisters the callback for disconnect.
Constraint	N/A
Parameters	@sockID: Socket ID.
Return value	Return 0 On success; return other On failure.

6.5.22 aic_socket_register_receive_ext_notify

Prototype	int aic_socket_register_receive_ext_notify(int sockID, void (*tcp_recv_ext_notify)(INT sock_id, int length, void *pdata), void *pdata)
Function	This function is extended receive callback to registers with BSD socket.
Constraint	N/A

AICXTEK Confidential

Parameters	@sockID: Socket ID. @tcp_recv_ext_notify: Extended Receive Callback API. @pdata: Extended parameters.
Return value	Return 0 On success; return other On failure.

6.5.23 aic_socket_unregister_receive_ext_notify

Prototype	int aic_socket_unegister_receive_ext_notify(int sockID)
Function	This function unregisters the callback for receive.
Constraint	N/A
Parameters	@sockID: Socket ID.
Return value	Return 0 On success; return other On failure.

6.5.24 aic_socket_register_tcp_ack_notify

Prototype	int aic_socket_register_tcp_ack_notify(int sockID, void (* tcp_ack_notify)(INT sock_id, int length, void *pdata), void *pdata)
Function	This function is tcp ack callback to registers with BSD socket.
Constraint	N/A
Parameters	@sockID: Socket ID. @tcp_ack_notify: ACK Callback API. @pdata: Extended parameters.
Return value	Return 0 On success; return other On failure.

6.5.25 aic_socket_unregister_tcp_ack_notify

Prototype	int aic_socket_unregister_tcp_ack_notify(int sockID)
Function	This function unregisters the callback for tcp ack.
Constraint	N/A
Parameters	@sockID: Socket ID.
Return value	Return 0 On success; return other On failure.

6.5.26 aic_socket_register_udp_drop_notify

Prototype	int aic_socket_register_udp_drop_notify(int sockID, void (* udp_drop_notify)(INT sock_id, int length, void *pdata), void *pdata)

AICXTEK Confidential

Function	This function is udp drop callback to registers with BSD socket.
Constraint	N/A
Parameters	@sockID: Socket ID. @udp_drop_notify: udp drop Callback API. @pdata: Extended parameters.
Return value	Return 0 On success; return other On failure.

6.5.27 aic_socket_unregister_udp_drop_notify

Prototype	int aic_socket_unregister_udp_drop_notify(int sockID)
Function	This function unregisters the callback for udp drop.
Constraint	N/A
Parameters	@sockID: Socket ID.
Return value	Return 0 On success; return other On failure.

6.5.28 aic_gethostbyname

Prototype	struct hostent *aic_gethostbyname(const char *name)
Function	This function is used for DNS parsing.
Constraint	N/A
Parameters	@name: pointer of hostname.
Return value	Return pointer of response On success; return NULL On failure.

6.6 AT command process api

Brief instruction

AT command process APIs include processing at commands received from the serial port,sending at commands to CP,handling response and URC received from CP etc..

Header file	Location
aic_at_process_common.h	aicos\components\include\at

6.6.1 aic_add_at_cmd_handle

Prototype	u32 aic_add_at_cmd_handle (AT_CMD_HANDLE *head_handle_ptr)
Function	Add at cmd handles for cmds received from the serial port and handled by AP.

AICXTEK Confidential

Constraint	N/A
Parameters	@head_handle_ptr: The pointer to the head AT_CMD_HANDLE.
Return value	Return AIC_SUCCESS if the handles are added OK,return other values if fail.
Demo	N/A

6.6.2 aic_add_at_resp_handle

Prototype	u32 aic_add_at_resp_handle(AT_RESP_ FROM_CP_HANDLE *head_handle_ptr)
Function	Add handles for handling response received from CP for certain at cmds.
Constraint	N/A
Parameters	@head_handle_ptr: The pointer to the head AT_RESP_ FROM_CP_HANDLE.
Return value	Return AIC_SUCCESS if the handles are added OK,return other values if fail.
Demo	N/A

6.6.3 aic_add_at_urc_handle

Prototype	u32 aic_add_at_urc_handle(AT_URC_ FROM_CP_HANDLE *head_handle_ptr)
Function	Add handles for handling URC with certain prefix received from CP.And the URC to handle doesn't contain the leading and ending \r\n.
Constraint	N/A
Parameters	@head_handle_ptr: The pointer to the head AT_URC_ FROM_CP_HANDLE.
Return value	Return AIC_SUCCESS if the handles are added OK,return other values if fail.
Demo	N/A

6.6.4 aic_register_cp_urc_callback

Prototype	u32 aic_register_cp_urc_callback (aic_get_cp_urc_callback cb)
Function	Register callback function to get all the URC received from CP. The URC string contains the leading and ending \r\n.

AICXTEK Confidential

Constraint	N/A
Parameters	@cb: The callback function.
Return value	Return AIC_SUCCESS if the callback is registered OK,return other values if fail.
Demo	N/A

6.6.5 aic_unregister_cp_urc_callback

Prototype	void aic_unregister_cp_urc_callback (aic_get_cp_urc_callback cb)
Function	Unregister callback function for URC received from CP.
Constraint	N/A
Parameters	@cb: The callback function.
Return value	N/A
Demo	N/A

6.6.6 aic_assemble_and_handle_at_cmd

Prototype	s32 aic_assemble_and_handle_at_cmd (u8 *at_str, ulong *at_req_resp, u8 is_external, u32 serial_channel_id, u8 retry_cnt)
Function	Assemble certain at cmd and send it to CP.Or handle at cmds received from the serial port.
Constraint	N/A
Parameters	@at_str: The complete at cmd string. @at_req_resp: The pointer to the pointer of AT_REQ_RESP struct packaging the at cmd. @is_external: The at cmd is received from external device or not. @serial_channel_id: The id of the serial port.If the cmd is sent by AP,that is internal,then the port id is ignored. @retry_cnt: The times to retry if time expires when receive response from CP.
Return value	Return AIC_SUCCESS if assemble and handle the cmd OK,return other values if fail.
Demo	N/A

6.6.7 aic_send_cmd_with_callback

AICXTEK Confidential

Prototype	u32 aic_send_cmd_with_callback(u8 *at_str, aic_get_cp_resp_callback cb)
Function	Assemble and send certain at command to CP.The response of the command will be informed by the callback function.
Constraint	N/A
Parameters	@at_str: The complete at cmd string. @cb: The callback function to get the response. It can be null.
Return value	Return AIC_SUCCESS if assemble and send the cmd OK,return other values if fail.
Demo	N/A

6.6.8 aic_send_cmd_and_wait

Prototype	u32 aic_send_cmd_and_wait(u8 *at_str, AT_RESP_FROM_CP *resp_to_save)
Function	Assemble and send certain at command to CP. And get the response in input parameter resp_to_save. The API is synchronized, so it will wait until receive response.
Constraint	N/A
Parameters	@at_str: The complete at cmd string. @resp_to_save: The struct to save the response. It can be null if not needed.
Return value	Return AIC_SUCCESS if assemble and send the cmd OK,return other values if fail.
Demo	N/A

6.6.9 aic_fill_transmit_cmd_resp

Prototype	s32 aic_fill_transmit_cmd_resp (AT_REQ_RESP *at_req_resp, ulong *handle_result, u16 line_cnt)
Function	Format and transmit the reply of certain at cmd to the upper device via the serial port.
Constraint	N/A
Parameters	@at_req_resp: The pointer to the struct AT_REQ_RESP packaging the at cmd. @handle_result: The pointer to the pointers of all reply lines. @line_cnt: The count of reply lines.

AICXTEK Confidential

Return value	Return AIC_SUCCESS if fill and transmit the reply lines OK,return other values if fail.
Demo	N/A

6.6.10 aic_reply_error_resp

Prototype	s32 aic_reply_error_resp (AT_REQ_RESP *at_req_resp_ptr, s32 error_value)
Function	Format and reply the error response to the upper device via the serial port.
Constraint	N/A
Parameters	@at_req_resp_ptr: The pointer to the struct AT_REQ_RESP packaging the at cmd. @error_value: The error number.
Return value	Return AIC_SUCCESS if format and transmit the error response OK,return other values if fail.
Demo	N/A

6.6.11 aic_reply_ok_resp

Prototype	s32 aic_reply_ok_resp (AT_REQ_RESP *at_req_resp_ptr)
Function	Format and reply response “OK” to the upper device via the serial port.
Constraint	N/A
Parameters	@at_req_resp_ptr: The pointer to the struct AT_REQ_RESP packaging the at cmd.
Return value	Return AIC_SUCCESS if format and transmit the “OK” response OK,return other values if fail.
Demo	N/A

6.6.12 aic_transmit_at_data

Prototype	s32 aic_transmit_at_data(u8 serial_port_id, u8 *data_buf, u16 data_len)
Function	Transmit the data payload to the upper device via the serial port.
Constraint	N/A
Parameters	@serial_port_id: The id of the serial port. It

AICXTEK Confidential

	can be got from AT_REQ_RESP struct. @data_buf: The pointer to the data buffer. @data_len: The length of the data buffer.
Return value	Return AIC_SUCCESS if transmit the data OK,return other values if fail.
Demo	N/A

6.6.13 aic_transmit_at_cmd_urc

Prototype	s32 aic_transmit_at_cmd_urc (u8 serial_port_id, u8 *str_cmd_urc, u16 urc_size)
Function	Transmit the URC of certain at cmd handled by AP to the upper device via the serial port.
Constraint	N/A
Parameters	@serial_port_id: The id of the serial port. It can be got from AT_REQ_RESP struct. @str_cmd_urc: The pointer to the URC string. @urc_size: The length of the URC string.
Return value	Return AIC_SUCCESS if transmit the URC string OK,return other values if fail.
Demo	N/A

6.6.14 aic_get_each_line_from_resp

Prototype	ulong *aic_get_each_line_from_resp(u8 *at_resp, u16 line_count)
Function	Get each line of the response received from CP for certain at cmd.
Constraint	N/A
Parameters	@at_resp: The pointer to the response received from CP. @line_count: The count of the response lines.
Return value	Return the pointer to the pointers of all lines if get OK,return NULL if fail.
Demo	N/A

6.6.15 aic_split_paras_to_str_array

Prototype	ulong *aic_split_paras_to_str_array(u8 *raw_str, u8 *start_str, u16 *paras_cnt)
Function	Get all parameters separated by commas from

AICXTEK Confidential

	raw string,the string can be an at cmd, one line of response or URC.
Constraint	N/A
Parameters	@raw_str: The raw string to extract parameters. @start_str: The starting string before the parameters. @paras_cnt: The pointer to the count of the parameters.
Return value	Return the pointer to the pointers of all parameter strings if get OK,return NULL if fail.
Demo	N/A

6.6.16 aic_set_at_handle_mode

Prototype	void aic_set_at_handle_mode (AT_REQ_RESP *at_req_resp, u8 at_mode)
Function	Set current mode for the at command being handled. If the at is waiting for data,then it should be set to AT_DATA_MODE,or the mode is AT_CMD_MODE.
Constraint	N/A
Parameters	@at_req_resp: The pointer to the struct AT_REQ_RESP packaging the at command. @at_mode: The mode for the at command handling.
Return value	N/A
Demo	N/A

6.6.17 aic_free_paras_mem

Prototype	void aic_free_paras_mem(ulong *ptr, u16 paras_cnt)
Function	Free the memory of all parameters or line strings extracted.
Constraint	It should be called after aic_split_paras_to_str_array or aic_get_each_line_from_resp is used.
Parameters	@ptr: The pointer to the pointers of all parameter or line strings. @paras_cnt: The count of the parameter or line strings.

AICXTEK Confidential

Return value	N/A
Demo	N/A

6.7 Transport Layer Security (TLS)

uses mbedtls library by default

Briefly instruction

API of TLS protocol(Depend on netxduo security library, which is closed by default).
Currently, these API only support client mode, connect/send/recv/select APIs reuse sockets.

Header file	Location
aic_tls_api.h	aicos\components\network\common\inc

6.7.1 aic_create_tls_socket

Prototype	int aic_create_tls_socket (INT protocolFamily, INT type, INT protocol)
Function	Create TLS socket & TLS session
Constraint	N/A
Parameters	@protocolFamily: Protocol family e.g AF_INET @type: Only TCP is supported @protocol: Socket protocol
Return value	Return socket descriptor if create OK,return error if fail to create.

6.7.2 aic_close_tls_socket

Prototype	int aic_close_tls_socket (u32 socket_id)
Function	Close TLS socket&delete TLS session
Constraint	N/A
Parameters	@ socket_id: Socket descriptor.
Return value	Return AIC_SUCCESS if close OK,return other value if fail.

6.7.3 aic_tls_ca_certificate_add

Prototype	Int aic_tls_ca_certificate_add (u32 socket_id, char * certificate, int certificate_len)
Function	Add TLS CA certificate
Constraint	N/A
Parameters	@ socket_id: Socket descriptor. @ certificate: pointer to certificate.

AICXTEK Confidential

	@ certificate_len: length of certificate.
Return value	Return AIC_SUCCESS if add TLS certificate success,return other values if fail to add.

6.7.4 aic_tls_ca_certificate_remove

Prototype	Int aic_tls_ca_certificate_remove (u32 socket_id, char * common_name, int common_name _len)
Function	remove TLS CA certificate
Constraint	N/A
Parameters	@ socket_id: Socket descriptor. @ common_name: pointer to common name. @ common_name _len: length of common name.
Return value	Return AIC_SUCCESS if remove TLS certificate success,return other values if fail to remove.

6.7.5 aic_tls_local_certificate_add

Prototype	Int aic_tls_ca_certificate_add (u32 socket_id, char * certificate, int certificate_len, char *private_key, int key_len, int key_type)
Function	Add TLS local certificate
Constraint	N/A
Parameters	@ socket_id: Socket descriptor. @ certificate: pointer to certificate. @ certificate_len: length of certificate. @ private_key: pointer to paivate key. @ key_len: length of paivate key. @ key_type: type of paivate key.
Return value	Return AIC_SUCCESS if add TLS certificate success,return other values if fail to add.

6.7.6 aic_tls_local_certificate_remove

Prototype	Int aic_tls_local_certificate_remove (u32 socket_id, char * common_name, int common_name _len)
Function	remove TLS local certificate
Constraint	N/A
Parameters	@ socket_id: Socket descriptor. @ common_name: pointer to common name. @ common_name _len: length of common

AICXTEK Confidential

	name.
Return value	Return AIC_SUCCESS if remove TLS certificate success,return other values if fail to remove.

6.7.7 aic_tls_ciphersuite_set

Prototype	Int aic_tls_ciphersuite_set (u32 socket_id, USHORT aic_ tls_ciphersuite)
Function	Specifying a TLS ciphersuite
Constraint	If you want to support all default ciphersuites, this interface does not need to be called.
Parameters	@ socket_id: Socket descriptor. @ aic_ tls_ciphersuite: TLS ciphersuite.
Return value	Return AIC_SUCCESS if remove TLS certificate success,return other values if fail to remove.

6.7.8 aic_tls_session_protocol_version_override

Prototype	Int aic_tls_session_protocol_version_override (u32 socket_id, u16 protocol_version)
Function	Set TLS socket protocol version
Constraint	Need to be called before connect.
Parameters	@ socket_id: Socket descriptor. @ protocol_version: TLS protocol version.
Return value	Return AIC_SUCCESS if set success,return other values if fail to set.

6.7.9 aic_secure_tls_session_renegotiate

Prototype	Int aic_secure_tls_session_renegotiate (u32 socket_id, u32 wait_option)
Function	Set TLS session renegotiate
Constraint	N/A
Parameters	@ socket_id: Socket descriptor. @ wait_option: timeout.
Return value	Return AIC_SUCCESS if set success,return other values if fail to set.

6.7.10 aic_secure_tls_session_time_function_set

Prototype	Int aic_secure_tls_ session_time_function_set (u32 socket_id, ulong (*time_func_ptr)(void))
Function	This function sets up a function pointer that TLS will invoke when it needs to get the current time, which is used in various TLS

AICXTEK Confidential

	handshake messages and for verification of certificates.
Constraint	N/A
Parameters	@ socket_id: Socket descriptor. @ time_func_ptr: pointer to time function.
Return value	Return AIC_SUCCESS if set success,return other values if fail to set.

6.7.11 aic_tls_set_session_sni_extension

Prototype	Int aic_tls_set_session_sni_extension (u32 socket_id, s8 *dns_url)
Function	This function initializes an X509 DNS name for use with extensions that use domain name entries.
Constraint	N/A
Parameters	@ socket_id: Socket descriptor. @ dns_url: dns name string.
Return value	Return AIC_SUCCESS if set success,return other values if fail to set.

6.8 Domain Name System(DNS)

Brief instruction

The APIs are for the function of domain name querying.

Header file	Location
aic_net_api.h	aicos\components\network\common\inc

6.8.1 aic_clear_dns_cache

Prototype	void aic_clear_dns_cache(void)
Function	Clear the results of domain name querying saved in the dns cache.
Constraint	N/A
Parameters	N/A
Return value	N/A
Demo	N/A

6.9 Access Point Name(APN)

AICXTEK Confidential

Brief instruction

The APIs are used to add and get APN info etc..

Header file	Location
aic_net_api.h	aicos\components\network\common\inc

6.9.1 aic_add_additional_apn

Prototype	s32 aic_ add_additional_apn (APN_INFO_ADDITIONAL *added_apns)
Function	Add additional apns except for the fixed apns.The fixed apns include the apns of operators from Chinese mainland,Hong Kong and Macao.
Constraint	N/A
Parameters	@ added_apns: The pointer of the apns to add.
Return value	Return AIC_SUCCESS if the apns are added OK,return other values if fail.
Demo	N/A

6.9.2 aic_get_fixed_apn_by_plmn

Prototype	s32 aic_get_fixed_apn_by_plmn (u8 *matched_plmn, u8 *matched_apn_type, APN_INFO *fixed_apn_ptr)
Function	Get the fixed APN_INFO by the plmn and apn type,such as the APN_INFO of plmn 46002 and default type.
Constraint	N/A
Parameters	@matched_plmn: The plmn to match. @matched_apn_type: The apn type to match. @fixed_apn_ptr: The pointer of the APN INFO to save in.
Return value	Return AIC_SUCCESS if get the corresponding fixed APN_INFO,return other values if fail.
Demo	N/A

6.9.3 aic_get_additional_apn_by_plmn

Prototype	s32 aic_get_additional_apn_by_plmn (u8 *matched_plmn, u8 *matched_apn_type,

AICXTEK Confidential

	CARRIER_APN *additional_apn_ptr)
Function	Get the additional CARRIER_APN by the plmn and apn type. The apn info is added via the API aic_ add_additional_apn.
Constraint	N/A
Parameters	@matched_plmn: The plmn to match. @matched_apn_type: The apn type to match. @additional_apn_ptr: The pointer of the CARRIER_APN to save in.
Return value	Return AIC_SUCCESS if get the corresponding CARRIER_APN,return other values if fail.
Demo	N/A

6.9.4 aic_get_apn_name_by_plmn

Prototype	void aic_get_apn_name_by_plmn (u8 *matched_plmn, u8 *matched_apn_type, u8 *apn_name)
Function	Get the APN name by the plmn and apn type.For example,get the APN name “cmnet” of plmn 46000 and default type.
Constraint	N/A
Parameters	@matched_plmn: The plmn to match. @matched_apn_type: The apn type to match. @apn_name: The pointer to save the APN name.
Return value	N/A
Demo	N/A

6.9.5 aic_get_apn_auth_type

Prototype	u8 aic_get_apn_auth_type (u8 *matched_plmn, u8 *matched_apn_type)
Function	Get the authentication type in apn info by the plmn and apn type.
Constraint	N/A
Parameters	@matched_plmn: The plmn to match. @matched_apn_type: The apn type to match.
Return value	Return the authentication type of 0,1 or 2 if found, return the invalid value 0xff if fail.
Demo	N/A

AICXTEK Confidential

7 NV

Brief instruction

NVRAM use for cp data manager, and keep data after power off. Config file contains configurations nvram use id.

Header file	Location
aic_nvram_api.h	aicos\components\nvram\cp\inc

Config file	Location
aic_nvram_id.h	Board: cp\common_inc

7.1 aic_nvram_get_length

Prototype	u16 aic_nvram_get_length(u16 item_id)
Function	Get nvram id length.
Constraint	N/A
Parameters	@item_id: nvram item id.
Return value	nvram id length

7.2 aic_nvram_read

Prototype	int aic_nvram_read(u16 item_id, u16 item_size, void *buf)
Function	Read nvram id data.
Constraint	N/A
Parameters	@item_id: nvram item id. @item_size: nvram item size. @buf: read buf.
Return value	AIC_SUCCESS is read pass, others fail.

7.3 aic_nvram_write

Prototype	int aic_nvram_write(u16 item_id, u16 item_size, const void *buf)
Function	Write nvram id data.
Constraint	N/A

AICXTEK Confidential

Parameters	@item_id: nvram item id. @item_size: nvram item size. @buf: write buf.
Return value	AIC_SUCCESS is write pass, others fail.

7.4 aic_nvram_sync_write

Prototype	int aic_nvram_sync_write(u16 item_id, u16 item_size, const void *buf, boolean forcesync)
Function	Write nvram id data.
Constraint	N/A
Parameters	@item_id: nvram item id. @item_size: nvram item size. @buf: write buf. @ forcesync: wait for write flash success then return if forcesync is true.
Return value	AIC_SUCCESS is sync write pass, others fail.

7.5 aic_nvram_flush

Prototype	int aic_nvram_flush(void)
Function	Flush nvram to flash.
Constraint	N/A
Parameters	N/A
Return value	AIC_SUCCESS is flush pass, others fail.

7.6 aic_nvram_restore_factory

Prototype	int aic_nvram_restore_factory(void)
Function	Nvram restore factory settings.
Constraint	N/A
Parameters	N/A
Return value	N/A

8 Miscdata

Brief instruction

Miscdata is used for ap data manager, and keep data after power off. Config file contains configurations miscdata use id.

AICXTEK Confidential

Header file	Location
aic_md_api.h	aicos\components\miscdata\inc

Config file	Location
aic_md_id.h	Board: ap\common_inc

8.1 aic_md_get_length

Prototype	u16 aic_md_get_length(u16 item_id)
Function	Get miscdata id length.
Constraint	N/A
Parameters	@item_id: miscdata item id.
Return value	miscdata id length

8.2 aic_md_read

Prototype	int aic_md_read(u16 item_id, u16 size, void *buf)
Function	Read miscdata id data.
Constraint	N/A
Parameters	@item_id: miscdata item id. @size: miscdata item size. @buf: read buf.
Return value	AIC_SUCCESS is read pass, others fail.

8.3 aic_md_write

Prototype	int aic_md_write(u16 item_id, u16 size, const void *buf)
Function	Write miscdata id data.
Constraint	N/A
Parameters	@item_id: miscdata item id. @size: miscdata item size. @buf: write buf.
Return value	AIC_SUCCESS is write pass, others fail.

8.4 aic_md_delete

Prototype	int aic_md_delete(u16 item_id)
Function	Delete miscdata id.
Constraint	N/A

AICXTEK Confidential

Parameters	@item_id: miscdata item id.
Return value	AIC_SUCCESS is delete pass, others fail.

8.5 aic_md_restore_factory

Prototype	void aic_md_restore_factory(void)
Function	Miscdata restore factory settings.
Constraint	N/A
Parameters	N/A
Return value	N/A.

9 LCD

Brief instruction

Display hardware init and refresh framebuffer to display panel.

Header file	Location
display.h	drivers\include

9.1 display_refresh_sync

Prototype	void aic_lcdc_panel_refresh_sync(u32 fb_addr)
Function	flip framebuffer data to panel as sync mode, no matter what system config mode.
Constraint	N/A
Parameters	@fb_addr: framebuffer address which to be flipped to panel display.
Return value	N/A

9.2 display_refresh

Prototype	void aic_lcdc_panel_refresh(u32 fb_addr)
Function	flip framebuffer data to panel as system config mode.(async or sync)
Constraint	N/A
Parameters	@fb_addr: framebuffer address which to be flipped to panel display.
Return value	N/A

9.3 display_suspend

Prototype	void display_suspend(void)
Function	take display ip & lcd panel into sleep, to save power

AICXTEK Confidential

Constraint	N/A
Parameters	N/A
Return value	N/A

9.4 display_resume

Prototype	void display_resume(void)
Function	take display ip & lcd panel from sleep, return normal work
Constraint	N/A
Parameters	N/A
Return value	N/A

9.5 aic_bl_brightness_set

Prototype	int aic_bl_brightness_set(u32 brt_perc)
Function	set lcd panel backlight brightness
Constraint	N/A
Parameters	@fb_addr:percentage of maximum brightness.
Return value	N/A

9.6 aic_bl_brightness_get

Prototype	int aic_bl_brightness_get(u32 *brt_perc)
Function	get lcd panel backlight brightness
Constraint	N/A
Parameters	@fb_addr:point to percentage of maximum brightness.
Return value	N/A

9.7 register_backlight_switch_monitor

Prototype	int register_backlight_switch_monitor(char *monitor_name, bl_switch_func callback);
Function	register backlight status change monitor
Constraint	N/A
Parameters	@monitor_name: user defined monitor name. @callback: backlight status change call monitor callback.
Return value	Success return zero.

9.8 unregister_backlight_switch_monitor

AICXTEK Confidential

Prototype	int unregister_backlight_switch_monitor(char *monitor_name);
Function	unregister backlight status change monitor.
Constraint	N/A
Parameters	@monitor_name: user defined monitor name.
Return value	Success return zero.

10 Camera

Brief instruction

The subsection introduces camera configure files, hal interfaces and jpeg compress&decompress interfaces.

10.1 Camera configure files

10.1.1 Header files

Header file	Location
camera_fb_cfg.h	aicos\projects\genesis\ap\xxx\config\
camera_cfg.h	aicos\projects\genesis\ap\xxx\config\

10.1.1.1 camera_fb_cfg.h

Prototype	CAMERA_MEM_SIZE
Function	Define the total memory size for camera framebuffers.
Constraint	Every framebuffer must be aligned at 4Kbyte size. If there are N framebuffers, it must be aligned at N * 4Kbyte.

Prototype	CAMERA_BUF_NUM
Function	Define the total number of camera framebuffers.
Constraint	NA.

10.1.1.2 camera_cfg.h

Prototype	CAMERA_OUTPUT_[x]LINE
Function	Define camera data lines number.
Constraint	User no need to modify this macro, current configuration is CAMERA_OUTPUT_2LINE.

Prototype	CAMERA_OUTPUT_[xxx]
Function	Define camera sensor data sync format.
Constraint	The support configuration is either

AICXTEK Confidential

	CAMERA_OUTPUT_MTK or CAMERA_OUTPUT_Z.

Prototype	CAMERA_[xxx]_MODE
Function	Define camera sensor data sample format.
Constraint	Current configuration is CAMERA_DDR_MODE, user should not modify it.

Prototype	struct aic_camera_cfg {
	IRQn_Type irq;
	GPIO_PIN_ID gpio_pwdn;
	GPIO_PIN_ID gpio_rst;
	void *cs_sensor_ops;
	#if defined(CONFIG_PMIC)
	struct pmu_user *vddcamio1v8;
	struct pmu_user *vddcama2v8;
	#endif
	};

Function	Used to define a camera sensor instance.
Constraint	The member irq should be set as CSI_IRQ, cs_sensor_ops should set as NULL, ther other members should set depend on Hardware.

10.2 Camera hal

Header file	Location
camera_hal.h	aicos\mm\camera\

10.2.1 camera_open

Prototype	int camera_open(camera_id_e id)
Function	Init a camera instance and open it.
Constraint	N/A
Parameters	@id: camera id
Return value	0: OK <0: ERROR number.

10.2.2 camera_close

Prototype	void camera_close(camera_id_e id)
Function	Close camera.
Constraint	N/A
Parameters	@id: camera id
Return value	void

AICXTEK Confidential

10.2.3 camera_preview

Prototype	int camera_preview(camera_id_e id, yuv_plane_num_e plane_num, camera_data_fmt_e data_fmt, camera_output_size_e image_size);
Function	Enter camera preview mode.
Constraint	N/A
Parameters	@id: camera id @plane_num: YUV_ONE_PLANE or YUV_TWO_PLANE for yuv422 @data_fmt: only support YUV(YUV422)now. @image_size: image size type
Return value	0: OK <0: ERROR number.

10.2.4 camera_capture_prepare

Prototype	int camera_capture_prepare(camera_id_e id, yuv_plane_num_e plane_num, camera_data_fmt_e data_fmt, camera_output_size_e image_size);
Function	Prepare for camera capture
Constraint	N/A
Parameters	@id: camera id @plane_num: YUV_ONE_PLANE or YUV_TWO_PLANE for yuv422 @data_fmt: only support YUV(YUV422)now. @image_size: image size type
Return value	0: OK <0: ERROR number.

10.2.5 Camera_capture

Prototype	int camera_capture(camera_id_e id, u32 user_addr);
Function	Entetr camera capture mode.
Constraint	N/A
Parameters	@id: camera id @user_addr: addr to save capture image.
Return value	0: OK <0: ERROR number.

10.2.6 camera_set_fps

Prototype	int camera_set_fps(camera_id_e id, u8 fps)
Function	Empty function for GX31X, do nothing now.

AICXTEK Confidential

Constraint	N/A
Parameters	@id: camera id @fps: frame rate
Return value	0: OK

10.2.7 camera_get_fps

Prototype	u8 camera_get_fps(camera_id_e id)
Function	Just return sensor frame rate 30 now.
Constraint	N/A
Parameters	@id: camera id
Return value	30

10.2.8 camera_ioctl

Prototype	int camera_ioctl(camera_id_e id, camera_cmd_e cmd, void *input, void *output);
Function	Camera ioctl.
Constraint	N/A
Parameters	@id: camera id @cmd: CAM_STREAM_ON or CAM_STREAM_OFF for GX31X. @input: input paramters, invalid for GX31X now @output: output parameters, invalid for GX31X now
Return value	0: OK <0: ERROR number.

10.3 Jpeg interface

Header file	Location
jpeglib.h	aicos\mm\camera\jpeg\jpeg-9e\

10.3.1 jpeg_compress_struct

Prototype	struct jpeg_compress_struct refer to jpeglib.h
Function	Abstruction of compress object
Constraint	NA

10.3.2 struct_decompress_struct

Prototype	struct jpeg_decompress_struct refer to jpeglib.h
Function	Abstruction of decompress object

AICXTEK Confidential

Constraint	NA

10.3.3 jpeg_create_compress

Prototype	void jpeg_create_compress(j_compress_ptr cinfo)
Function	Create a compress object
Constraint	NA
Parameters	@ cinfo: pointer to the compress object
Return value	void

10.3.4 jpeg_std_error

Prototype	struct jpeg_error_mgr * jpeg_std_error (struct jpeg_error_mgr * err)
Function	Default error-management setup
Constraint	Should be set before compress start.
Parameters	@ err: error-management object pointer
Return value	Return the error-management object pointer

10.3.5 jpeg_stdio_dest

Prototype	void jpeg_stdio_dest (j_compress_ptr cinfo, FILE * outfile)
Function	Set compress object destination to file
Constraint	The destination file should be opened before calling the function, and closed after compress done.
Parameters	@ cinfo: pointer to the compress object @outfile: the destination file pointer
Return value	void

10.3.6 jpeg_mem_dest

Prototype	void jpeg_mem_dest (j_compress_ptr cinfo, unsigned char ** outbuffer, size_t * outsize)
Function	Set compress object destination to memory.
Constraint	NA
Parameters	@cinfo: pointer to the compress object @outbuffer: pointer to the compressed data @outsize: size of the compressed data
Return value	void

10.3.7 jpeg_set_defaults

Prototype	void jpeg_set_defaults (j_compress_ptr cinfo)
Function	Default parameter setup for compression

AICXTEK Confidential

Constraint	NA
Parameters	@cinfo: pointer to the compress object
Return value	Void

10.3.8 jpeg_start_compress

Prototype	void jpeg_start_compress (j_compress_ptr cinfo, boolean write_all_tables)
Function	Main entry points for compression
Constraint	Before calling this, all parameters and a data destination must be set up.
Parameters	@cinfo: pointer to the compress object @write_all_tables: Suggest set true for failsafe check
Return value	void

10.3.9 jpeg_write_raw_data

Prototype	JDIMENSION jpeg_write_raw_data (j_compress_ptr cinfo, JSAMPIMAGE data, JDIMENSION num_lines)
Function	Replaces jpeg_write_scanlines when writing raw downsampled data.
Constraint	NA
Parameters	@cinfo: pointer to the compress object @data: the component data to be written @num_lines: number of scanlines
Return value	0: suspended lines_per_iMCU_row: one iMCU row

10.3.10 jpeg_write_scanlines

Prototype	JDIMENSION jpeg_write_scanlines (j_compress_ptr cinfo, JSAMPARRAY scanlines, JDIMENSION num_lines)
Function	Write some scanlines of data to the JPEG compressor.
Constraint	This should be less than the supplied num_lines only in case that the data destination module has requested suspension of the compressor, or if more than image_height scanlines are passed in.
Parameters	@cinfo: pointer to the compress object @scanlines: pointer to the scanlines that will be written to JPEG compressor @num_lines: number of the scanlines to be written
Return value	The return value will be the number of lines actually written.

AICXTEK Confidential

10.3.11 jpeg_finish_compress

Prototype	void jpeg_finish_compress (j_compress_ptr cinfo)
Function	Finish JPEG compression.
Constraint	NA
Parameters	@cinfo: pointer to the compress object
Return value	void

10.3.12 jpeg_destroy_compress (j_compress_ptr cinfo)

Prototype	void jpeg_destroy_compress (j_compress_ptr cinfo)
Function	Destruction of a JPEG compression object
Constraint	NA
Parameters	@cinfo: pointer to the compress object
Return value	void

10.3.13 jpeg_abort_compress

Prototype	void jpeg_abort_compress (j_compress_ptr cinfo)
Function	Abort processing of a JPEG compression operation, but don't destroy the object itself.
Constraint	NA
Parameters	@cinfo: pointer to the compress object
Return value	void

10.3.14 jpeg_create_decompress

Prototype	void jpeg_create_decompress(j_decompress_ptr cinfo)
Function	Create a decompress object
Constraint	NA
Parameters	@ j_decompress_ptr: pointer to the decompress object
Return value	void

10.3.15 jpeg_stdio_src

Prototype	void jpeg_stdio_src (j_decompress_ptr cinfo, FILE * infile)
Function	Prepare for input from a stdio stream.
Constraint	The caller must have already opened the stream, and is responsible for closing it after finishing decompression.
Parameters	@cinfo: pointer to the decompress object @infile: source file handle
Return value	void

10.3.16 jpeg_read_header

Prototype	int jpeg_read_header(j_compress_ptr cinfo,

AICXTEK Confidential

	boolean require_image)
Function	Decompression startup: read start of JPEG datastream to see what's there.
Constraint	Need only initialize JPEG object and supply a data source before calling.
Parameters	@cinfo: pointer to the decompress object @require_image: when set true, you need not check for TABLES_ONLY return code
Return value	JPEG_SUSPENDED: Suspended due to lack of input data JPEG_HEADER_OK: Found valid image datastream JPEG_HEADER_TABLES_ONLY: Found valid table-specs-only datastream

10.3.17 jpeg_start_decompress

Prototype	Boolean jpeg_start_decompress (j_decompress_ptr cinfo)
Function	Decompression initialization.
Constraint	jpeg_read_header must be completed before calling this.
Parameters	@cinfo: pointer to the decompress object
Return value	Returns FALSE if suspended. The return value need be inspected only if a suspending data source is used.

10.3.18 Jpeg_read_scanlines

Prototype	JDIMENSION jpeg_read_scanlines (j_decompress_ptr cinfo, JSAMPARRAY scanlines, JDIMENSION max_lines)
Function	Read some scanlines of data from the JPEG decompressor.
Constraint	we warn about excess calls to jpeg_read_scanlines() since this likely signals an application programmer error. However, an oversize buffer (max_lines > scanlines remaining) is not an error.
Parameters	@cinfo: pointer to the decompress object @scanlines: pointer to lines to be read @max_lines: max number of lines to be read
Return value	The return value will be the number of lines actually read. This may be less than the number requested in several cases, including bottom of image, data source suspension, and operating modes that emit multiple scanlines at a time.

10.3.19 Jpeg_finish_decompress

Prototype	boolean jpeg_finish_decompress (j_decompress_ptr cinfo)
Function	Finish JPEG decompression.
Constraint	NA

AICXTEK Confidential

Parameters	@cinfo: pointer to the decompress object
Return value	Returns FALSE if suspended. The return value need be inspected only if a suspending data source is used.

10.3.20 jpeg_destroy_decompress

Prototype	void jpeg_destroy_decompress (j_decompress_ptr cinfo)
Function	Destruction of a JPEG decompression object
Constraint	NA
Parameters	@cinfo: pointer to the decompress object
Return value	void

10.3.21 jpeg_abort_decompress

Prototype	void jpeg_abort_decompress (j_decompress_ptr cinfo)
Function	Abort processing of a JPEG decompression operation, but don't destroy the object itself.
Constraint	NA
Parameters	@cinfo: pointer to the decompress object
Return value	void

11 Audio

Brief instruction

Audio stream control, volume and mute etc..

Header file	Location
audio_system.h	mm\audio

11.1 enum astream_type

Prototype	enum astream_type { ASTREAM_TYPE_MUSIC, ASTREAM_TYPE_RING, ASTREAM_TYPE_SYSTEM, ASTREAM_TYPE_TONE, ASTREAM_TYPE_DTMF, ASTREAM_TYPE_VOICE, ASTREAM_TYPE_VOIP, ASTREAM_TYPE_FM, ASTREAM_TYPE_TTS, ASTREAM_TYPE_ALARM, ASTREAM_TYPE_NOTIFICATION,

AICXTEK Confidential

	ASTREAM_TYPE_RECORD, ASTREAM_TYPE_VOICE_UL, ASTREAM_TYPE_VOIP_UL, ASTREAM_TYPE_MAX, };
Function	Enum of audio stream type.
Constraint	Not all stream type is supported in a specific platform.

11.2 enum aplay_type

Prototype	enum aplay_type {
	APLAY_TYPE_NONE = 0,
	APLAY_TYPE_BUF,
	APLAY_TYPE_FILE,
	APLAY_TYPE_ID, /* Not suppport*/
	APLAY_TYPE_TONE,
	APLAY_TYPE_TTS,
	APLAY_TYPE_STREAM, /* such like online play */
	APLAY_TYPE_MAX
	};

Function	Audio play type, which means how or where we get audio stream data from.
Constraint	Not all types are supported now.

11.3 enum astream_format

Prototype	enum astream_format {
	ASTREAM_FORMAT_INVALID = -1,
	ASTREAM_FORMAT_DEFAULT = 0,
	ASTREAM_FORMAT_PCM,
	ASTREAM_FORMAT_WAV,
	ASTREAM_FORMAT_MP3,
	ASTREAM_FORMAT_AAC,
	ASTREAM_FORMAT_AMR_NB,
	ASTREAM_FORMAT_AMR_WB,
	ASTREAM_FORMAT_SBC,
	ASTREAM_FORMAT_MIDI,
	ASTREAM_FORMAT_MAX
	}

Function	Audio stream format.
Constraint	Not all format are supported now.

AICXTEK Confidential

11.4 enum astream_source

Prototype	enum astream_source {
	ASTREAM_SOURCE_DEFAULT = 0,
	ASTREAM_SOURCE_MIC,
	ASTREAM_SOURCE_VOICE_UPLINK,
	ASTREAM_SOURCE_VOICE_DONWLINK,
	ASTREAM_SOURCE_FM,
	ASTREAM_SOURCE_BT_SCO,
	ASTREAM_SOURCE_MAX
	};

Function	Enum of audio source, audio stream come from in record scenario.
Constraint	Not supported now.

11.5 enum astream_channel

Prototype	enum astream_channel {
	ASTREAM_CHN_NONE = 0,
	ASTREAM_CHN_MONO = 1,
	ASTREAM_CHN_STEREO = 2,
	ASTREAM_CHN_MAX
	};

Function	PCM stream channel count, if compress format(mp3,aac,amr), it will be ignored.
Constraint	N/A

11.6 enum astream_direct

Prototype	enum astream_direct {
	ASTREAM_DIR_PLAY = 0,
	ASTREAM_DIR_RECORD = 1,
	};

Function	Audio stream direction.
Constraint	N/A

11.7 enum audio_mode

Prototype	enum audio_mode { AUDIO_MODE_NORMAL = 0, AUDIO_MODE_RING, AUDIO_MODE_CALL, AUDIO_MODE_COMMUNICATE, AUDIO_MODE_MAX

AICXTEK Confidential

	};
Function	Enum of audio mode, it determines which device will be used to play audio.
Constraint	Not all modes are supported now.

11.8 enum audio_device

Prototype	enum audio_device {
	AUDIO_DEVICE_NONE = 0x0u,
	AUDIO_DEVICE_BIT_IN = 0x80000000u,
	AUDIO_DEVICE_BIT_DEFAULT = 0x40000000u,
	AUDIO_DEVICE_OUT_EARPIECE = 0x1u,
	AUDIO_DEVICE_OUT_SPEAKER = 0x2u,
	AUDIO_DEVICE_OUT_HEADSET = 0x4u,
	AUDIO_DEVICE_OUT_HEADPHONE = 0x8u,
	AUDIO_DEVICE_OUT_BT_SCO = 0x10u,
	AUDIO_DEVICE_OUT_BT_A2DP = 0x20u,
	AUDIO_DEVICE_OUT_FM = 0x40u,
	AUDIO_DEVICE_IN_MIC = 0x80000001u,
	AUDIO_DEVICE_IN_HEADPHONE = 0x80000002u,
	AUDIO_DEVICE_IN_BACK_MIC = 0x80000004u,
	AUDIO_DEVICE_IN_BT_SCO = 0x80000008u,
	AUDIO_DEVICE_IN_LINE = 0x80000020u,
	};

Function	Enum of audio device.
Constraint	Not all devices are supported now.

11.9 enum audio_output_flag

Prototype	enum audio_output_flag {
	AUDIO_OUTPUT_FLAG_NONE = 0x0,
	AUDIO_OUTPUT_FLAG_DIRECT = 0x1,
	AUDIO_OUTPUT_FLAG_FAST = 0x2,
	AUDIO_OUTPUT_FLAG_DEEP_BUFF = 0x4,
	AUDIO_OUTPUT_FLAG_COMPRESS = 0x8,
	AUDIO_OUTPUT_FLAG_INCALL_ULINK = 0x10,
	AUDIO_OUTPUT_FLAG_INCALL_DLINK = 0x20,
	AUDIO_OUTPUT_FLAG_SYNC = 0x40
	};

Function	Audio output flag.
Constraint	Only AUDIO_OUTPUT_FLAG_DIRECT and AUDIO_OUTPUT_FLAG_COMPRESS are supported now.

AICXTEK Confidential

11.10 enum astream_cb_event

Prototype	enum astream_cb_event {
	ASTREAM_CB_EVENT_NONE = 0,
	ASTREAM_CB_EVENT_FINISH,
	ASTREAM_CB_EVENT_BREAK,
	};

Function	Enum of application callback event.
Constraint	N/A

11.11 enum tone_type

Prototype	enum tone_type { /* DTMF tones ITU-T Recommendation Q.23 */ TONE_DTMF_0 = 0, TONE_DTMF_1, TONE_DTMF_2, TONE_DTMF_3, TONE_DTMF_4, TONE_DTMF_5, TONE_DTMF_6, TONE_DTMF_7, TONE_DTMF_8, TONE_DTMF_9, TONE_DTMF_S, TONE_DTMF_P, TONE_DTMF_A, TONE_DTMF_B, TONE_DTMF_C, TONE_DTMF_D, /* Call supervisory tones: 3GPP TS 22.001 (CEPT) */ TONE_SUP_DIAL, FIRST_SUP_TONE = TONE_SUP_DIAL, TONE_SUP_BUSY, TONE_SUP_CONGESTION, TONE_SUP_RADIO_ACK, TONE_SUP_RADIO_NOTAVAIL, TONE_SUP_ERROR, TONE_SUP_CALL_WAITING, TONE_SUP_RINGTONE, LAST_SUP_TONE = TONE_SUP_RINGTONE, /* Proprietary tones: 3GPP TS 31.111 */ TONE_PROP_BEEP,

AICXTEK Confidential

	TONE_PROP_ACK, TONE_PROP_NACK, TONE_PROP_PROMPT, TONE_PROP_BEEP2, };
Function	Enum of tone type.
Constraint	N/A

11.12 astream_user_cb

Prototype	typedef int (*astream_user_cb)(u32 data, enum astream_cb_event event);
Function	Application callback function.
Constraint	N/A
Parameters	@data: user registed data @astream_cb_event event: callback event
Return value	0: OK <0: ERROR number.

11.13 audio_open_output

Prototype	ACLIENT audio_open_output(enum astream_type type, enum aplay_type aplay_type, enum astream_format format, u32 flag);
Function	Open audio output that to play music.
Constraint	N/A
Parameters	@type:audio stream type. @aplay_type:play type. @flag: steam flag.
Return value	>0: handle =0: no handle, open failed.

11.14 audio_open_input

Prototype	ACLIENT audio_open_input(enum aplay_type aplay_type, enum astream_format format, u32 sample_rate, enum astream_channel chn, enum astream_source source, u32 flag);
Function	Open audio input that to record.
Constraint	N/A
Parameters	@type:audio stream type. @ format:recorded file format. @ sample_rate:sample rate @ chn: audio channel number

AICXTEK Confidential

	@ source: where record come from @flag: steam flag.
Return value	>0: handle =0: no handle, open failed.

11.15 audio_open

Prototype	ACLIENT audio_open(enum astream_type type, enum aplay_type aplay_type, u32 flag, u32 sample_rate, u32 bit_rate, enum astream_format format, enum astream_channel chn, enum astream_direct dir, enum astream_source source);
Function	Open audio stream control handle.
Constraint	N/A
Parameters	@type:audio stream type. @aplay_type:play type. @flag: steam flag. @sample_rate: sample rate. @bit_rate: bit rate. @format: format. @chn: channel count. @dir: stream direction. @source: where record come from.
Return value	0: OK <0: ERROR number.

11.16 audio_play

Prototype	int audio_play(ACLIENT handle);
Function	Start audio play or record by handle.
Constraint	N/A
Parameters	@handle: gotten by audio_open.
Return value	0: OK <0: ERROR number.

11.17 audio_stop

Prototype	int audio_stop(ACLIENT handle);
Function	Stop audio play or record.

AICXTEK Confidential

Constraint	N/A
Parameters	@handle: gotten by audio_open.
Return value	0: OK <0: ERROR number.

11.18 audio_close

Prototype	int audio_close(ACLIENT handle);
Function	Close audio handle.
Constraint	N/A
Parameters	@handle: gotten by audio_open.
Return value	0: OK <0: ERROR number.

11.19 audio_play_buf

Prototype	int audio_play_buf(ACLIENT handle, char *buf, u32 buf_len, void *user_cb, u32 cb_data);
Function	Play audio data of given buffer.
Constraint	N/A
Parameters	@handle: gotten by audio_open. @buf: data buffer of audio data. @buf_len: data buffer length. @user_cb: functoin to be called when playing is stopped or interrupted. @cb_data: parameters of user_cb.
Return value	0: OK <0: ERROR number.

11.20 audio_play_file

Prototype	int audio_play_file(ACLIENT handle, char *file_name, void *user_cb, u32 cb_data);
Function	Play audio by file name.
Constraint	Not support if no file system.
Parameters	@handle: gotten by audio_open. @file_name: file name.

AICXTEK Confidential

	@user_cb: functoin to be called when playing is stopped or interrupted. @cb_data: parameters of user_cb.
Return value	0: OK <0: ERROR number.

11.21 audio_play_stream

Prototype	int audio_play_stream(ACLIENT handle, void *user_cb, u32 cb_data);
Function	Play stream type audio, caller should fill data with audio_write.
Constraint	N/A
Parameters	@handle: gotten by audio_open. @user_cb:callback function, used to notify caller play finished or broken etc. @cb_data:callback function data.
Return value	0: OK <0: ERROR number.

11.22 audio_play_tone

Prototype	int audio_play_tone(enum tone_type tone_type, u32 duation);
Function	Play tone, need not open a handle.
Constraint	N/A
Parameters	@tone_type: tone type. @duation: duration of tone play.
Return value	0: OK <0: ERROR number.

11.23 audio_pause

Prototype	int audio_pause(ACLIENT handle);
Function	Pause audio play by handle.
Constraint	N/A
Parameters	@handle: gotten by audio_open.
Return value	0: OK <0: ERROR number.

11.24 audio_resume

AICXTEK Confidential

Prototype	int audio_resume(ACLIENT handle);
Function	Resume paused audio stream by handle.
Constraint	N/A
Parameters	@handle: gotten by audio_open.
Return value	0: OK <0: ERROR number.

11.25 audio_read

Prototype	int audio_read(ACLIENT handle, char *buf, u32 len, u32 wait_time);
Function	read audio stream data.
Constraint	N/A
Parameters	@handle: gotten by audio_open. @buf:audio data buffer. @len:length of buffer. @wait_time:maximum wait time in ms.
Return value	>=0: size of getten data. <0: ERROR number.

11.26 audio_write

Prototype	int audio_write(ACLIENT handle, char *buf, u32 len, u32 wait_time);
Function	Write audio stream data.
Constraint	N/A
Parameters	@handle: gotten by audio_open. @buf:audio data buffer. @len:audio data length in buffer. @wait_time:maximum wait time in ms.
Return value	0: OK <0: ERROR number.

11.27 audio_drain

Prototype	int audio_drain(ACLIENT handle);

AICXTEK Confidential

Function	Notify audio system all data is written.
Constraint	N/A
Parameters	@handle: gotten by audio_open.
Return value	0: OK <0: ERROR number.

11.28 audio_force_out_device

Prototype	int audio_force_out_device(u32 device);
Function	Force set out device, like speaker, headset etc.
Constraint	N/A
Parameters	@device: out device want to set.
Return value	0: OK <0: ERROR number.

11.29 audio_force_in_device

Prototype	int audio_force_in_device(u32 device);
Function	Force set in device, like main mic, headset mic etc.
Constraint	N/A
Parameters	@device:
Return value	0: OK <0: ERROR number.

11.30 audio_get_device

Prototype	u32 audio_get_device(void);
Function	Get current device, both in and out.
Constraint	N/A
Parameters	N/A
Return value	Current device.

11.31 audio_get_force_device

Prototype	u32 audio_get_force_device(void);
Function	Get forced device, both in and out.
Constraint	N/A
Parameters	N/A
Return value	Forced device.

11.32 audio_set_volume

Prototype	int audio_set_volume(ACLIENT handle,

AICXTEK Confidential

	u32 vol);
Function	Set audio volume by handle.
Constraint	N/A
Parameters	@handle: gotton by audio_open. @vol: volume level want to set.
Return value	0: OK <0: ERROR number.

11.33 audio_get_volume

Prototype	u32 audio_get_volume(ACLIENT handle)
Function	Get audio volume by handle.
Constraint	N/A
Parameters	@handle: gotten by audio_open.
Return value	Volume level.

11.34 audio_set_master_volume

Prototype	int audio_set_master_volume(u32 vol)
Function	Set audio system volume.
Constraint	N/A
Parameters	@vol: volume level want to set.
Return value	0: OK <0: ERROR number.

11.35 audio_get_master_volume

Prototype	u32 audio_get_master_volume(void)
Function	Get audio system volume.
Constraint	N/A
Parameters	N/A
Return value	Volume level

11.36 audio_mute

Prototype	int audio_mute(ACLIENT handle)
Function	Mute audio stream by handle.
Constraint	N/A
Parameters	@handle: getten by audio_open.
Return value	0: OK <0: ERROR number.

11.37 audio_unmute

Prototype	int audio_unmute(ACLIENT handle)

AICXTEK Confidential

Function	Unmute audio stream by handle.
Constraint	N/A
Parameters	@handle: gotten by audio_open.
Return value	0: OK <0: ERROR number.

11.38 audio_mute_all

Prototype	int audio_mute_all(void);
Function	Mute audio system.
Constraint	N/A
Parameters	N/A
Return value	0: OK <0: ERROR number.

11.39 audio_unmute_all

Prototype	int audio_unmute_all(void);
Function	Unmute audio system.
Constraint	N/A
Parameters	N/A
Return value	0: OK <0: ERROR number.

12 Cache

Brief instruction

There are 32KB data cache and 32KB instruction cache in GX31X mcu core system, both cache line size are 32 bytes.

Header files

Header file	Location
cache.h	drivers\include

12.1 cache_enable

Prototype	void cache_enable(void)
Function	Enable data cache and instruction cache.
Constraint	N/A
Parameters	N/A
Return value	N/A

12.2 cache_disable

AICXTEK Confidential

Prototype	void cache_disable(void)
Function	Disable data cache and instruction cache.
Constraint	N/A
Parameters	N/A
Return value	N/A

12.3 cache_invcc_all

Prototype	void cache_invcc_all(void)
Function	Clean and invalidate the whole data cache and instruction cache area.
Constraint	N/A
Parameters	N/A
Return value	N/A

12.4 icache_enable

Prototype	void icache_enable(void)
Function	Enable instruction cache.
Constraint	N/A
Parameters	N/A
Return value	N/A

12.5 icache_disable

Prototype	void icache_disable(void)
Function	Disable instruction cache.
Constraint	N/A
Parameters	N/A
Return value	N/A

12.6 icache_inv_by_range

Prototype	void icache_inv_by_range(u32 addr, size_t length)
Function	Invalidate icache by range.
Constraint	The addr should be cache line size aligned, if not , warning info will be output and will be rounded down to cache line size aligned; The length should be integer multiples of cache line size, or warning info will be output and will be rounded down to integer multiples of cache line size.
Parameters	@addr: start address to be invalidated in

AICXTEK Confidential

	instruction cache. @length: length in bytes to be invalidated in instruction cache.
Return value	N/A

12.7 icache_inv_all

Prototype	void icache_inv_all(void)
Function	Invalidate the whole instruction cache area.
Constraint	N/A
Parameters	N/A
Return value	N/A

12.8 dcache_enable

Prototype	void dcache_enable(void)
Function	Enable data cache.
Constraint	N/A
Parameters	N/A
Return value	N/A

12.9 dcache_disable

Prototype	void dcache_disable(void)
Function	Disable data cache.
Constraint	N/A
Parameters	N/A
Return value	N/A

12.10 dcache_clean_by_range

Prototype	void dcache_clean_by_range(u32 addr, size_t length)
Function	Clean data cache by range.
Constraint	The addr should be cache line size aligned, if not , warning info will be output and will be rounded down to cache line size aligned; The length should be integer multiples of cache line size, or warning info will be output and will be rounded down to integer multiples of cache line size.
Parameters	@addr: start address to be cleaned in data cache. @length: length in bytes to be cleaned in data

AICXTEK Confidential

	cache.
Return value	N/A

12.11 dcache_clean_all

Prototype	void dcache_clean_all(void)
Function	Clean all data cache.
Constraint	N/A
Parameters	N/A
Return value	N/A

12.12 dcache_inv_by_range

Prototype	void dcache_inv_by_range(u32 addr, size_t length)
Function	Invalidate data cache by range.
Constraint	The addr should be cache line size aligned, if not , warning info will be output and will be rounded down to cache line size aligned; The length should be integer multiples of cache line size, or warning info will be output and will be rounded down to integer multiples of cache line size.
Parameters	@addr: start address to be invalidated in data cache. @length: length in bytes to be invalidated in data cache.
Return value	N/A

12.13 dcache_inv_all

Prototype	void dcache_inv_all(void)
Function	Invalidate the whole data cache area.
Constraint	N/A
Parameters	N/A
Return value	N/A

12.14 dcache_invcc_by_range

Prototype	void dcache_invcc_by_range(u32 addr, size_t length)
Function	Clean and invalidate data cache by range.
Constraint	The addr should be cache line size aligned, if

AICXTEK Confidential

	not , warning info will be output and will be rounded down to cache line size aligned; The length should be integer multiples of cache line size, or warning info will be output and will be rounded down to integer multiples of cache line size.
Parameters	@addr: start address to be cleaned and invalidated in data cache. @length: length in bytes to be cleaned and invalidated in data cache.
Return value	N/A

12.15 dcache_invcc_all

Prototype	void dcache_invcc_all(void)
Function	Clean and invalidate the whole data cache area.
Constraint	N/A
Parameters	N/A
Return value	N/A

13 MPU

Brief instruction

In an implementation that include Protected Memory System Architecture(PMSA), system address space is protected by a Memory Protection Unit(MPU). The GX31X mcu mpu supports up to 16 regions which can be configured flexible.

Enum

Enum	Description
MPU_AP_T	NACCESS: 0, access not allowed for a region RDONLY: 1, read only for a region RDWR: 2, both read and write for a region
MPU_CBTEX_T	DEV_NGNRNE: 0, access the range of a region as device MEM_NC: 1, access the range of a region as normal memory, non cacheable MEM_WTNWA: 2, access the range of a region as normal memory, with write through, read allocate cache policy MEM_WBNWA: 3, access the range of a region as normal memory, with write back , read

AICXTEK Confidential

	allocate cache policy MEM_WBWA: 4, access the range of a region as normal memory, with write back, read/write allocate cache policy
MPU_XN_T	NO_EXEC: 0, execution is not allowed for a region EXEC: 1, execution is allowed for a region
MPU_CONFIG_T	CONF_EXCLUDE_RDONLY: 0, CONF_APPEND_RDONLY: 1, CONF_NORMAL: 2,

Struct

Struct	Description
MPU_REGION_T	The MPU_REGION_T type is the whole attributes of a mpu region. @base_addr: start address of a region. @end_addr: end address of a region. @ap: access privilege of a region. @cbtex: access attributes and cache policy of a region. @xn: execution attribute of a region. @rg_stat: enable/disable a region.

Header files

Header file	Location
mpu.h	drivers\include

13.1 mpu_dynamic_conf_region

Prototype	int mpu_dynamic_conf_region(const MPU_REGION_T *rg)
Function	Configure a mpu region with specified settings.
Constraint	N/A
Parameters	@rg: The pointer of the specified region configuration.
Return value	Return negative number if failed, or return the region index which no less than 0.

13.2 mpu_dynamic_unconf_region

Prototype	void mpu_dynamic_unconf_region(int index)
Function	Unconfig a mpu region with the region index returned by mpu_dynamic_conf_region

AICXTEK Confidential

	function. This function will disable and release the region.
Constraint	N/A
Parameters	@index: The index of the region that get from mpu_dynamic_conf_region function.
Return value	N/A

14 DMA

Brief instruction

There are 22 DMA channels, the master IP which support DMA mode can use all the 22 DMA channels, and software DMA memory copy can also use one of these channels.

Header file	Location
dma.h	drivers\include

14.1 aic_dma_chn_request

Prototype	int aic_dma_chn_request(DMA_CALLBACK cb, void *param)
Function	Request one DMA channel.
Constraint	N/A
Parameters	@cb: DMA callback function. If DMA transfer done, it will be called by DMA interrupt function. @param: this is input parameter of callback function.
Return value	Return DMA channel number if success, or return AIC_ENODEV.

14.2 aic_dma_chn_free

Prototype	void aic_dma_chan_free(u32 chn)
Function	Free the DMA channel which previously requested.
Constraint	N/A
Parameters	@chn: DMA channel number.
Return value	N/A

14.3 aic_dma_ll_cfg

Prototype	int aic_dma_ll_cfg(u32 chn,

AICXTEK Confidential

	u32 llp, struct dma_chan_cfg *cfg, u32 node_cnt)
Function	Configure DMA linklist nodes.
Constraint	N/A
Parameters	@chn:declare the DMA channel number which need to be configured as linklist mode. @llp: the linklist node physical address. @cfg: the configuration parameter struct pointer. @node_cnt: declare how many linklist nodes needs to configure.
Return value	Should return AIC_SUCCESS always.

14.4 aic_dma_chn_cfg

Prototype	int aic_dma_chn_cfg(u32 chn, struct dma_chn_cfg *cfg)
Function	Configure DMA normal mode.
Constraint	N/A
Parameters	@chn:declare the DMA channel number which need to be configured. @cfg: the configuration parameter struct pointer.
Return value	Should return AIC_SUCCESS always.

14.5 aic_dma_chn_start

Prototype	void aic_dma_chn_start(u32 chn)
Function	Start DMA transfer.
Constraint	N/A
Parameters	@chn: DMA channel number.
Return value	N/A

14.6 aic_dma_chn_get_src

Prototype	u32 aic_dma_chn_get_src(u32 chn)
Function	Get DMA channel current source address.
Constraint	N/A
Parameters	@chn: declare the DMA channel number.
Return value	Return DMA current source address.

14.7 aic_dma_chn_get_dst

AICXTEK Confidential

Prototype	u32 aic_dma_chn_get_dst(u32 chn)
Function	Get DMA channel current desternation address.
Constraint	N/A
Parameters	@chn: declare the DMA channel number.
Return value	Return DMA current source address.

15 Clock

Briefly instruction

Clock types are divived into fixed clocks, mux clocks, divider clocks, composite clocks and gate clocks.
Fixed clocks contain 26M、32K and PLLs. The mux clock have multiple parent clocks. Each divider clock only have one parent clock, it's frequency can be divided by parent clock. The composite clock not only has multiple parent clocks, but also it's frequency can be divided by parent clock. Gate clocks are used as switches.

Header file	Location
aic_iot_clk.h	drivers\include
aic_clk_map.h	projects\genesis\ap\common_inc\chip_reg

15.1 clk_set_parent_mux

Prototype	Int clk_set_parent_mux(u8 clk_id, u8 index)
Function	Change mux clock's parent.
Constraint	The clk_id must be mux clock id, index should not be overflow.
Parameters	@clk_id: mux clock id. @index: mux clock parent index.
Return value	0

15.2 clk_set_parent_comp

Prototype	Int clk_set_parent_comp(u8 clk_id, u8 index)
Function	Change composite clock's parent.
Constraint	The clk_id must be composite clock id, index should not be overflow.
Parameters	@clk_id: composite clock id. @index: composite clock parent index.

AICXTEK Confidential

Return value	0

15.3 clk_set_rate_comp

Prototype	Int clk_set_rate_comp(u8 clk_id, u32 rate)
Function	Change composite clock's rate.
Constraint	The clk_id must be composite clock id, rate should not be overflow.
Parameters	@clk_id: composite clock id. @rate: the rate to be set.
Return value	0

15.4 clk_set_rate_div

Prototype	Int clk_set_rate_div(u8 clk_id, u32 rate)
Function	Change divider clock's rate.
Constraint	The clk_id must be composite clock id, rate should not be overflow.
Parameters	@clk_id: composite clock id. @rate: the rate to be set.
Return value	0

15.5 clk_enable

Prototype	Int clk_enable(u8 clk_id)
Function	Enable gate clock.
Constraint	The clk_id must be gate clock id.
Parameters	@clk_id: gate clock id.
Return value	0

15.6 clk_disable

Prototype	Int clk_disable(u8 clk_id)
Function	Disable gate clock.
Constraint	The clk_id must be gate clock id.
Parameters	@clk_id: gate clock id.
Return value	0

15.7 clk_get_parent_mux

Prototype	u8 clk_get_parent_mux(u8 clk_id)

AICXTEK Confidential

Function	Get mux clock's current parent clock id.
Constraint	The clk_id must be mux clock id.
Parameters	@clk_id: mux clock id.
Return value	0

15.8 clk_get_parent_comp

Prototype	u8 clk_get_parent_comp(u8 clk_id)
Function	Get composite clock's current parent clock id.
Constraint	The clk_id must be composite clock id.
Parameters	@clk_id: composite clock id.
Return value	0

15.9 clk_get_rate_comp

Prototype	u32 clk_get_rate_comp(u8 clk_id)
Function	Get composite clock's current rate.
Constraint	The clk_id must be composite clock id.
Parameters	@clk_id: composite clock id.
Return value	0

15.10 clk_get_rate_mux

Prototype	u32 clk_get_rate_mux(u8 clk_id)
Function	Get mux clock's current rate.
Constraint	The clk_id must be mux clock id.
Parameters	@clk_id: mux clock id.
Return value	0

15.11 clk_get_rate_div

Prototype	u32 clk_get_rate_div(u8 clk_id)
Function	Get divider clock's current rate.
Constraint	The clk_id must be divider clock id.
Parameters	@clk_id: divider clock id.
Return value	0

16 IPU

Brief instruction

There are 30 interrupts which can wakeup ap system from deepsleep.

Header file	Location
ipu_inc.h	aicos\drivers\include

AICXTEK Confidential

16.1 IPU_IRQEnable

Prototype	void IPU_IRQEnable(u32 value)
Function	Enable specific intrrupt.
Constraint	N/A
Parameters	@value: bit value of interrupt‘s index.
Return value	N/A

16.2 IPU_IRQDisable

Prototype	void IPU_IRQDisable(u32 value)
Function	Disable specific intrrupt .
Constraint	N/A
Parameters	@value: bit value of interrupt‘s index.
Return value	N/A

17 EIPU

Brief instruction

There are 8 eipu channels and support dbnc/latch/async/sync types.

Header file	Location
eipu.h	aicos\drivers\include

17.1 EIPU_RegCallback

Prototype	int EIPU_RegCallback(u32 chan_id, EIPU_FUNC_TYPE_E eipu_type, EIPU_CALLBACK callback, EIPU_TRG_TYPE_E trg_type_e, u32 dbnc_cnt_ms, EIPU_MODE_TYPE_E sec_mode);
Function	Register specific eipu channel to the system,which support external interrupt source.
Constraint	N/A
Parameters	@chan_id: specific channel number. @eipu_type: type of eipu. @callback: callback from the eipu’s irq. @trg_type_e: type of trigger. @dbnc_cnt_ms: timing for dbnc type. @sec_mode:mode of subsystem.
Return value	Should return AIC_SUCCESS always.

AICXTEK Confidential

17.2 EIPU_SetPol

Prototype	void EIPU_SetPol(EIPU_MODE_TYPE_E sec_mode, EIPU_FUNC_TYPE_E eipu_type, u32 chan_id, u32 pol_val);
Function	Set eipu channel's polarity value.
Constraint	N/A
Parameters	@sec_mode:mode of subsystem. @eipu_type: type of eipu. @chan_id: specific channel number. @pol_val: polarity of initial trigger level.
Return value	Should return AIC_SUCCESS always.

17.3 EIPU_GetIndata

Prototype	u32 EIPU_GetIndata(EIPU_MODE_TYPE_E sec_mode, EIPU_FUNC_TYPE_E eipu_type, u32 chan_id);
Function	Get inputdate of specific channel.
Constraint	N/A
Parameters	@sec_mode:mode of subsystem @eipu_type: type of eipu @chan_id: specific channel number
Return value	Should return AIC_SUCCESS always.

18 Timer

Brief instruction

There is 32KHz timer, it includes 32KHz system timer and freerun timer.

Header file	Location
timer_inc.h	drivers\include

18.1 aic_get_tick

Prototype	u64 aic_get_tick(void)
Function	Get threadx system tick counter.
Constraint	N/A
Parameters	N/A
Return value	Return threadx system tick counter.

AICXTEK Confidential

18.2 aic_get_timestamp

Prototype	u32 aic_get_timestamp(void)
Function	Get chip timestamp counter.
Constraint	N/A
Parameters	N/A
Return value	Return chip timestamp counter.

18.3 aic_delay

Prototype	void aic_delay(u32 sec)
Function	This api used for second time delay.
Constraint	N/A
Parameters	@sec: declare the seconds couter need to be delayed.
Return value	N/A

18.4 aic_mdelay

Prototype	void aic_mdelay(u32 sec)
Function	This api used for millisecond time delay.
Constraint	N/A
Parameters	@sec: declare the millisecond couter need to be delayed.
Return value	N/A

18.5 aic_udelay

Prototype	void aic_udelay(u32 sec)
Function	This api used for microseconds time delay.
Constraint	N/A
Parameters	@sec: declare the microseconds couter need to be delayed.
Return value	N/A

19 UART

Brief instruction

Total six uart port,they are mcu uart0、mcu uart1、mcu uart2、aon uart、top uart0、top uart1. Config file contains configurations uart mode、buadrate、cd、dma、trigger level、hw flow ctrl, and etc.

Header file	Location
serial_inc.h	drivers\include

AICXTEK Confidential

Config file	Location
serial_config.h	Board: config\serial

19.1 uart_open

Prototype	void *uart_open (int com)
Function	Open uart com port.
Constraint	N/A
Parameters	@com: com port.
Return value	Open handle pointer.

19.2 uart_close

Prototype	void uart_close (void *handle)
Function	Close uart com port.
Constraint	N/A
Parameters	@handle: uart open handle pointer.
Return value	N/A

19.3 uart_read

Prototype	size_t uart_read (void *handle, void *buffer, size_t size)
Function	Read data from uart fifo.
Constraint	N/A
Parameters	@handle: uart open handle. @buffer: read buffer. @size: read size.
Return value	Actually read size.

19.4 uart_write

Prototype	size_t uart_write (void *handle, const void *buffer, size_t size)
Function	Write data to uart fifo.
Constraint	N/A
Parameters	@handle: uart open handle. @buffer: write buffer. @size: write size.
Return value	Actually write size.

AICXTEK Confidential

19.5 uart_set_baud_rate

Prototype	void uart_set_baud_rate (void *handle, int rate)
Function	Set uart com baund.
Constraint	N/A
Parameters	@handle: uart open handle. @rate: new baud rate.
Return value	N/A

20 I2C

Brief instruction

This chapter lists APIs which can be used to read/write data by I2C bus.

Header file	Location
i2c.h	drivers\include
Structures	description

struct i2c_client {
u8 addr;
u32 index;
void *data;
}
	A device is a client relative to a bus, and the software uses the client structure to describe device-related properties.
	Structure member introduction:
	@addr: device address.
	@index: indicates which bus is used.
	@data: the handle of the I2C bus.

20.1 i2c_open

Prototype	int i2c_open(struct i2c_client *client)
Function	The software associates the device with the bus.
Constraint	N/A
Parameters	@client:The structure client of the device description.
Return value	Should return AIC_EIO if open I2C fail, or return AIC_SUCCESS.

20.2 i2c_close

Prototype	void i2c_close(struct i2c_client *client)
Function	The software disconnects the device from the

AICXTEK Confidential

	bus.
Constraint	N/A
Parameters	@client: The structure client of the device description.
Return value	N/A.

20.3 i2c_master_send

Prototype	int i2c_master_send(struct i2c_client *client, u8*buf, int num)
Function	Send data by I2C bus.
Constraint	N/A
Parameters	@client: the structure client of the device description. @buf: space address of the data store. @num: the number of data to be sent.
Return value	The number of data actually sent.

20.4 i2c_master_recv

Prototype	int i2c_master_recv(struct i2c_client *client, u8*buf, int num)
Function	Receive data by I2C bus.
Constraint	N/A
Parameters	@client: The structure client of the device description. @buf: Space address of the data store. @num: the number of data that can be received.
Return value	The number of data actually received.

20.5 i2c_master_send_then_recv

Prototype	void i2c_master_send_then_recv(struct i2c_client *client, u8 *addr, u16 addr_len, u8*buf, int num)
Function	Receive data by the I2C bus to the specified address.
Constraint	N/A

AICXTEK Confidential

Parameters	@client: the structure client of the device description. @addr: the register address of the device. @addr_len: the register address length. @buf: space address of the data store. @num: the number of data that can be received.
Return value	The number of data actually received.

20.6 i2c_transfer

Prototype	int i2c_transfer(struct i2c_client *client, struct i2c_msg *msgs, int num)
Function	Transfer data by I2C bus.
Constraint	N/A
Parameters	@client: The structure client of the device description. @msgs: the smallest unit of I2C driver transmission. @num: the number of MSG to be transferred.
Return value	Return the number of message actually transferred, or other return value when transferred fail

21 PMIC interface

Brief instruction

API used to communicate with power managment IC.

Header file	Location
pmic_io.h	drivers\include

21.1 pmic_writel

Prototype	int pmic_writel(u16 addr, u32 val)
Function	Write data to a specified address.
Constraint	N/A
Parameters	@addr: the address to access. @val: the data to be written.

AICXTEK Confidential

Return value	AOS_SUCCESS means success, the other values mean error.

21.2 pmic_readb

Prototype	U8 pmic_readb(u16 addr)
Function	Read a byte of data at the specified address.
Constraint	N/A
Parameters	@addr: the address to access.
Return value	Return one byte value read from the specified address

21.3 pmic_readl

Prototype	U32 pmic_readb(u16 addr)
Function	Read a word of data at the specified address.
Constraint	N/A
Parameters	@addr: the address to access.
Return value	Return one word value read from the specified address.

21.4 pmic_burst_write

Prototype	int pmic_burst_write(u8 *buf, int num)
Function	Write batch data continuously.
Constraint	N/A
Parameters	@buf: space address of the data store. The first two bytes of buffer store the destination address which data will be written to. @num: the number of data to be sent
Return value	AOS_SUCCESS means success, other values mean error.

21.5 pmic_burst_read

Prototype	int pmic_burst_read(u16 addr, u8 *buf, int num)
Function	Read batch data continuously.
Constraint	N/A
Parameters	@addr: the address to access. @buf: space address of data to be read.

AICXTEK Confidential

	@num: number of data that can be received
Return value	AOS_SUCCESS means success, other values mean error.

22 SSP

Brief instruction

SSP operate interface.

Header file	Location	
ssp.h	drivers\include	

Config file	Location	
ssp_config.h	Board: config\	

Structures	Description

struct ssp_cfg{
  enum spi_mode spi_mode;
  enum data_width data_width;
  u32 work_clk;
  u8 cs;
  slave_notify slv_tx_notify;
  slave_notify slv_rx_notify;
  void *slv_param;
}
	ssp configuration. Used for ssp device config, Include spi mode, data width, work clock, cs. slv_tx_notify/slv_rx_notify is optinal, they can be used to notify slave tx/rx is ready in slave mode. Slv_param is parameter for slv_tx_notify/slv_rx_notify.

Enums	**Description**

enum bus_proto {
  PROTO_SPI = 0,
  PROTO_SSP,
  PROTO_MW,
  PROTO_I2S,
  PROTO_I2S_DSP,
  PROTO_MAX
}
	define the bus protocal ssp controller supported.

enum spi_mode spi_mode {
  SPI_MODE_0 = 0,
  SPI_MODE_1,
  SPI_MODE_2,
  SPI_MODE_3,
  SPI_MODE_MAX
}
	define clock polarity and clock phase for spi mode.
	Mode0: CPOL = 0, CPHA = 0
	Mode1: CPOL = 0, CPHA = 1
	Mode2: CPOL = 1, CPHA = 0
	Mode3: CPOL = 1, CPHA = 1

AICXTEK Confidential

enum work_mode {
MASTER_MODE = 0,
SLAVE_MODE,
WORK_MODE_MAX
}
	define master/slave mode.

enum data_width {
DW_BYTE = 0,
DW_HWORD,
DW_WORD,
DW_MAX
}
	define bus data width.
	DW_BYTE: 8 bits
	DW_HWORD: 16 bits
	DW_WORD: 32 bits

enum active_lvl {
ACTIVE_LOW = 0,
ACTIVE_HIGH,
ACTIVE_LVL_MAX
}
	define the active level of cs.

22.1 ssp_device_register

Prototype	int ssp_device_register(u32 *handle, char *name, char *bus_name, struct ssp_cfg cfg,)
Function	Register ssp device to ssp bus with device name, bus name and cfg, and return handle if success. There are two ssp controller, we call them ssp bus, their name are “ssp0” and “ssp1”. Configuration contains spi mode, data width, work frequence, cs, and so on. See definition of struct ssp_cfg.
Constraint	N/A
Parameters	@handle: ssp device handle. @name: ssp device name. @bus_name: ssp bus name. @config: ssp config parameters.
Return value	Open success return zero.

22.2 ssp_read

Prototype	int ssp_read(u32 handle, u8 *buf, u32 size)
Function	Read data from bus.

AICXTEK Confidential

Constraint	N/A
Parameters	@handle: ssp device handle. @buf: the buffer to store read data. @size: the size to be read.
Return value	Return the actual length of read data.

22.3 ssp_write

Prototype	int ssp_write(u32 handle, u8 *buf, u32 size)
Function	Write data to bus.
Constraint	N/A
Parameters	@handle: ssp device handle. @buf: the buffer to store write data. @size: the size to be write.
Return value	Write success return zero.

22.4 ssp_write_read

Prototype	int ssp_write_read(u32 handle, u8 *wbuf, u8 *rbuf, u32 size)
Function	Write/Read data to/from bus.
Constraint	N/A
Parameters	@handle: ssp device handle. @wbuf: the buffer to store write data. @rbuf: the buffer to store read data. @size: the size to be transfer.
Return value	Write/Read success return zero.

22.5 ssp_write_then_read

Prototype	int ssp_write_then_read(u32 handle, u8 *wbuf, u32 wsize, u8 *rbuf, u32 rsize)
Function	Write data to bus, then read from bus.

AICXTEK Confidential

Constraint	N/A
Parameters	@handle: ssp device handle. @wbuf: the buffer to store write data. @wsize: the size to write. @rbuf: the buffer to store read data. @rsize: the size to be read.
Return value	If success return zero.

22.6 ssp_write_then_write

Prototype	int ssp_write_then_write(u32 handle, u8 *wbuf, u32 wsize u8 *wbuf1, u32 wsize1)
Function	Write data to bus, then write data1 to bus.
Constraint	N/A
Parameters	@handle: ssp device handle. @wbuf: the buffer to store write data. @wsize: the size to write. @wbuf1: the buffer to store write data. @wsize1: the size to write.
Return value	If success return zero.

23 USB

23.1 usb_get_connect_status

Prototype	BOOL usb_get_connect_status(void)
Function	Get the USB connection status.
Constraint	N/A
Parameters	N/A
Return value	TRUE: USB cable plug in. FALSE: USB cable plug out.

24 GPIO

Brief instruction

There are 48 GPIO pin in GX31X system, it includes 16 AON GPIO pin and 32 TOP GPIO pin.

AICXTEK Confidential

Header file	Location
gpio_inc.h	drivers\include

24.1 aic_gpio_output

Prototype	void aic_gpio_output(GPIO_PIN_ID index, GPIO_MODE mode)
Function	Configure GPIO pin output mode.
Constraint	N/A
Parameters	@index: declare GPIO pin which need to be configured. @mode: GPIO output mode, e.g. GPIO_OUTPUT_LVL_LOW, or GPIO_OUTPUT_LVL_HIGH.
Return value	N/A

24.2 aic_gpio_input

Prototype	void aic_gpio_input(GPIO_PIN_ID index, GPIO_MODE mode, GPIO_CBFUN gpio_cb, void *param)
Function	Configure GPIO input mode, and request input callback.
Constraint	N/A
Parameters	@index: declare GPIO pin which need to be configured. @mode: GPIO input mode, e.g. GPIO_TRIGGER_LVL_LOW, or GPIO_TRIGGER_LVL_HIGH. @gpio_cb: GPIO input callback function. @param: GPIO input callback function input parameter.
Return value	N/A

24.3 aic_gpio_level_get

Prototype	u32 aic_gpio_level_get(GPIO_PIN_ID index)
Function	GPIO pin input level status.
Constraint	N/A
Parameters	@index: declare GPIO pin which need to be read.

AICXTEK Confidential

Return value	N/A

Notice that！！
If the chip supports both GPIO and GPO, it can use gpio api to control GPO pin, so that the consumer can use same api of GPIO to control GPIO and GPO pins for compatibly.
In the following enumerations, GPO index is 100 offset.(file aic_gpio_pin.h)

```
typedef enum  {
   GPIO_0 = 0,
   GPIO_1,
   GPIO_2,
   GPIO_3,
   GPIO_4,
   GPIO_5,
   GPIO_6,
   GPIO_7,
   GPIO_8,
   GPIO_9,
   GPIO_10,
   GPIO_11,
   GPIO_12,
   GPIO_13,
   GPIO_14,
   GPIO_15,
   GPIO_16,
   GPIO_17,
   GPIO_18,
   GPIO_19,
   GPIO_20,
   GPIO_21,
   GPIO_22,
   GPIO_23,
   GPIO_24,
   GPIO_25,
   GPIO_26,
   GPIO_27,
   GPIO_28,
   GPIO_29,
   GPIO_30,
   GPIO_31,
   /* GPIO GPO index, phy gpo index in gpo_cfg.h */
   GPIO_GPO_0  = AIC_GPIO_GPO_MAPPING_ID,
   GPIO_GPO_1,
```

AICXTEK Confidential

GPIO_GPO_2,
GPIO_GPO_3,
GPIO_GPO_4,
GPIO_GPO_5,
GPIO_GPO_6,
GPIO_GPO_7,
GPIO_GPO_8,
GPIO_GPO_9,
GPIO_GPO_10,
GPIO_GPO_11,
GPIO_GPO_12,
GPIO_GPO_13,
GPIO_GPO_14,
GPIO_GPO_15,
GPIO_GPO_16,
GPIO_GPO_17,
GPIO_GPO_18,
GPIO_GPO_19,
GPIO_GPO_20,
GPIO_GPO_21,
GPIO_GPO_22,
GPIO_GPO_23,
GPIO_GPO_24,
GPIO_GPO_25,
GPIO_GPO_26,
GPIO_GPO_27,
GPIO_GPO_28,
GPIO_GPO_29,
GPIO_GPO_30,
GPIO_GPO_31,
GPIO_GPO_32,
GPIO_GPO_33,
GPIO_GPO_34,
GPIO_GPO_35,
GPIO_GPO_36,
GPIO_GPO_37,
GPIO_GPO_38,
GPIO_GPO_39,
GPIO_GPO_40,
GPIO_GPO_41,
GPIO_GPO_42,
GPIO_GPO_43,
GPIO_GPO_44,

AICXTEK Confidential

```
    GPIO_GPO_45,
    GPIO_GPO_46,
    GPIO_GPO_47,
    GPIO_GPO_48,
    GPIO_GPO_49,
    GPIO_GPO_50,
    GPIO_GPO_51,
    GPIO_GPO_52,
    GPIO_GPO_53,
    GPIO_GPO_54,
    GPIO_GPO_55,
    GPIO_GPO_56,
    GPIO_GPO_57,
    GPIO_GPO_58,
    GPIO_GPO_59,
    GPIO_GPO_60,
    GPIO_PIN_MAX
} GPIO_PIN_ID;
```

25 GPO

Brief instruction

GPO support output high/low level, but not support input mode. All the gpo index could be found in gpo_cfg.h file.

Header file	Location
gpo_inc.h	drivers\include

25.1 aic_gpo_output

Prototype	void aic_gpo_output(u32 gpo_id, GPO_MODE mode)
Function	Configure GPO pin output mode.
Constraint	N/A
Parameters	@gpo_id: the GPO pin which need to be requested. @mode: GPO output mode, e.g. GPO_OUTPUT_LOW, or GPO_OUTPUT_HIGH.
Return value	N/A

25.2 aic_gpo_get_level

Prototype	GPO_MODE aic_gpo_get_level(u32 pin)

AICXTEK Confidential

Function	GPO pin input level status.
Constraint	N/A
Parameters	@index: declare GPO pin which need to be read.
Return value	GPO output mode, e.g. GPO_OUTPUT_LOW, GPO_OUTPUT_HIGH, Or GPO_DISABLE

25.3 aic_gpo_pin_bare_cfg

Prototype	void aic_gpo_pin_bare_cfg(u32 pin_base, GPO_MODE mode)
Function	Configure GPO pin output mode.
Constraint	N/A
Parameters	@ pin_base: the GPO pin which need to be requested, it should be pin base address. @mode: GPO output mode, e.g. GPO_OUTPUT_LOW, or GPO_OUTPUT_HIGH.
Return value	N/A

26 Pin control

Brief instruction

The pins are configured together during the system initialization, it is nothing to do for developer normally. In some case, if the pins need to be reconfigure, just do this in corresponding pin_cfg array in pin_cfg.c which located in projects\genesis\ap\board_name\config

Header files

Header file	Location
pinlist.h	projects\genesis\ap\common_inc

27 Keypad

Briefly instruction

The keypad controller can detect 4 key events at a scan period. It also supports the detection of the ghost key. The keypad driver create a thread with a queue, when keypad controller detects one or more key events, it will send a message to the thread.

Header file	Location

AICXTEK Confidential

kpc.h	drivers\include

27.1 kpc_msg

Prototype	struct kpc_msg {
	u16 key0_code;
	u16 key0_event;
	u16 key1_code;
	u16 key1_event;
	u16 key2_code;
	u16 key2_event;
	u16 key3_code;
	u16 key3_event;
	};

Function	Used to transfer kpc event message

27.2 kpc_bisr_thrd_pid

Prototype	aos_thread_id kpc_bisr_thrd_pid

28 GPD

Brief instruction

There are 4 GPD channels, each channel can work independently.

Header file	Location
gpd_inc.h	drivers\include

28.1 aic_gpd_update

Prototype	void aic_gpd_update(GPD_INDEX index)
Function	Update GPD channel parameter, this channel will count repeatly.
Constraint	N/A
Parameters	@index: GPD channel.
Return value	N/A

28.2 aic_gpd_mode_cfg

Prototype	void aic_gpd_mode_cfg(GPD_INDEX index, GPD_EDGE_TYPE mode)
Function	Configure GPD channel mode.

AICXTEK Confidential

Constraint	N/A
Parameters	@index: GPD channel. @mode: configure GPD channel mode, e.g. GPD_POSEDGE_TYP, or GPD_NEGEDGE_TYPE, or GPD_BOTHEDGE_TYPE.
Return value	N/A

28.3 aic_gpd_cnt_cfg

Prototype	void aic_gpd_cnt_cfg(GPD_INDEX index, u32 cnt)
Function	Configure GPD channel counter.
Constraint	N/A
Parameters	@index: GPD channel. @cnt: GPD channel monitor counter.
Return value	N/A

28.4 aic_gpd_get_cnt

Prototype	u32 aic_gpd_get_cnt(GPD_INDEX index)
Function	Get GPD channel counter.
Constraint	N/A
Parameters	@index: GPD channel.
Return value	Return GPD channel counter.

28.5 aic_gpd_get_cycle

Prototype	u32 aic_gpd_get_cycle(GPD_INDEX index)
Function	Get GPD channel cycle counter.
Constraint	N/A
Parameters	@index: GPD channel.
Return value	Return GPD channel cycle counter.

29 PWM

Brief instruction

Pulse-width modulation operate interface. User scaling pwm frequence and duty ratio via the interface.

Header file	Location
pwm.h	drivers\include

29.1 pwm_config

AICXTEK Confidential

Prototype	int pwm_config(char* pwm_name, u32 duty_ns, u32 period_ns)
Function	Set pwm duty & period then output.
Constraint	N/A
Parameters	@pwm_name: pwm name, please reference these names in pwm_cfg.h @duty_ns: pwm duty time, duty time must less then or equal to period time, unit is ns. @period_ns: pwm period time, unit is ns.
Return value	None zero is pwm config fail, the cause is pwm name is invalid.

30 Watchdog

Brief instruction

Watchdog is used for monitor system hung up, afte start watchdog, a low priority task should be start to kick watchdog timer to prevent watchdog timeout. Watchdog have twice timeout, first timeout will raise the interrupt to cpu, second timeout will trigger reset request to PMIC then reset SoC.

Header file	Location
wdt.h	drivers\include

30.1 aic_watchdog_ping

Prototype	void aic_watchdog_ping(void)
Function	Kick aic watchdog.
Constraint	N/A
Parameters	N/A
Return value	N/A

30.2 aic_watchdog_start

Prototype	void aic_watchdog_start(void)
Function	Watchdog start.
Constraint	N/A
Parameters	N/A
Return value	N/A

30.3 aic_watchdog_stop

AICXTEK Confidential

Prototype	void aic_watchdog_stop(void)
Function	Watchdog stop.
Constraint	N/A
Parameters	N/A
Return value	N/A

30.4 aic_watchdog_set_timeout

Prototype	void aic_watchdog_set_timeout(u32 irq_timeout_ms, u32 rst_timeout_ms)
Function	Watchdog irq timeout time and reset timeout time config.
Constraint	N/A
Parameters	aic watchdog have two types of timeout time, 1. irq_timeout_ms: first timeout, generates an interruption to CPU, inform system wdt timeout handle. 2. rst_timeout_ms: irq timeout untreated, second timeout, generates an reset signal to pmic reset whole system.
Return value	N/A

30.5 aic_watchdog_mode_config

Prototype	void aic_watchdog_mode_config(AIC_WDT_MODE mode)
Function	Watchdog mode config.
Constraint	N/A
Parameters	@mode: aic watchdog have three types of work mode, 3. timeout irq only 4. timeout reset only 5. first time timeout irq, second time timeout trigger reset.
Return value	N/A

31 Power control

Brief instruction

Wakelock prevent subsystem entering deepsleep state. Sleephook support driver's callback the when system enter suspend/resume. System handle shutdown or reset.

Header file	Location
slp_inc.h	aicos\drivers\include

31.1 aic_get_wakelock

Prototype	int aic_get_wakelock(void);
Function	Get wakelock to prevent deepsleep.

AICXTEK Confidential

Constraint	N/A
ParametersN/A	N/A
Return value	Should return AIC_SUCCESS always.

31.2 aic_put_wakelock

Prototype	int aic_put_wakelock(void);
Function	Put wakelock to allow deepsleep.
Constraint	N/A
Parameters	N/A
Return value	Should return AIC_SUCCESS always.

31.3 slp_register_sleep_hook

Prototype	int slp_register_sleep_hook(SLP_HOOK_T *slp_hook);
Function	Register driver's suspend/resume callback to system
Constraint	N/A
Parameters	@slp_hook: ptr to slp callback and pararms.
Return value	Should return AIC_SUCCESS always.

31.4 slp_unregister_sleep_hook

Prototype	int slp_unregister_sleep_hook(SLP_HOOK_T *slp_hook);
Function	Remove driver's suspend/resume callback from system
Constraint	N/A
Parameters	@slp_hook: pointer to slp callback and pararms.
Return value	Should return AIC_SUCCESS always.

31.5 sysoff_shutdown

Prototype	void sysoff_shutdown()
Function	System will shutdown If using the same function in bootloader, please use bl_shutdown()
Constraint	N/A
Parameters	N/A
Return value	N/A

31.6 sysoff_reset

AICXTEK Confidential

Prototype	void sysoff_reset()
Function	System will reset, it is warm reset. If using the same function in bootloader, please use bl_reset()
Constraint	N/A
Parameters	N/A
Return value	N/A

31.7 aic_boot_mode_get

Prototype	BOOT_MODE_E aic_boot_mode_get()
Function	System will reset, it is warm reset
Constraint	1. This interface is in kernel, the same interface of enum BOOT_MOD get_boot_mode(void) is in bootloader. 2. Set mode is from sysoff_mode() in sysoff_manage.c or aic_boot_mode_set() in kernel
Parameters	N/A
Return value	N/A

31.8 sysoff_register_hook

Prototype	u32 sysoff_register_hook(SYSOFF_HOOK_FUNC sysoff_func, void* sysoff_param_ptr)
Function	It will handle registered function before system reset or shutdown. If unregister function, using sysoff_unregister_hook interface.
Constraint	N/A
Parameters	N/A
Return value	N/A

32 Efuse

Brief instruction

efuse block data operate interface, include each block date 32bit read and write.

Header file	Location
efuse.h	drivers\include

32.1 aic_efuse_block_write

Prototype	int aic_efuse_block_write(

AICXTEK Confidential

	int blk, u32 blk_val)
Function	Write 32bit data to assign efuse block.
Constraint	N/A
Parameters	@blk: efuse block num. @blk_val: the block value to be written.
Return value	Write success return zero.

32.2 aic_efuse_block_read

Prototype	int aic_efuse_block_read(int blk, u32 *blk_val)
Function	Read data from assign efuse block.
Constraint	N/A
Parameters	@blk: efuse block num. @blk_val: point to uint32 space used to store read back efuse block value.
Return value	Read success return zero.

33 Flash managment device

Brief instruction

Flash managment device(fmd) operate interface.

Header file	Location
fmd.h	drivers\include

33.1 fmd_get_partition_name

Prototype	char *fmd_get_partition_name(u32 part_id)
Function	Get fmd partition name by part id.
Constraint	N/A
Parameters	@part_id: partition id.
Return value	Partition name.

33.2 fmd_open

Prototype	int fmd_open(char *name, u32 *handle)
Function	Open fmd partition by name, and return handle if success.
Constraint	N/A

AICXTEK Confidential

Parameters	@name: fmd partition name. @handle: fmd partition handle.
Return value	Open success return zero.

33.3 fmd_close

Prototype	int fmd_close(u32 handle)
Function	Close fmd partition.
Constraint	N/A
Parameters	@handle: fmd partition handle.
Return value	Close success return zero.

33.4 fmd_read

Prototype	int fmd_read(u32 handle, u32 offset, u32 size, u8 *buf)
Function	Read data from partition.
Constraint	The specified offset and size must not be out of partition range.
Parameters	@handle: fmd partition handle. @offset: the address start to read in partition. @size: the size to be read. @buf: the buffer to store read data.
Return value	Read success return zero.

33.5 fmd_write

Prototype	int fmd_write(u32 handle, u32 offset, u32 size, u8 *buf)
Function	Write data to partiton.
Constraint	The specified offset and size must not be out of partition range.
Parameters	@handle: fmd partition handle. @offset: the address start to write in partition. @size: the size to be write. @buf: the buffer to store write data.
Return value	Write success return zero.

AICXTEK Confidential

33.6 fmd_erase

Prototype	int fmd_erase(u32 handle, u32 offset, u32 size)
Function	Erase partition.
Constraint	The specified offset and size must not be out of partition range. The size must align to minimal erase size which can be gained by fmd_get_erase_size interface.
Parameters	@handle: fmd partition handle. @offset: the address start to erase in partition. @size: the size to be erase.
Return value	Erase success return zero.

33.7 fmd_get_partition_size

Prototype	int fmd_get_partition_size(u32 handle, u32 *retsize)
Function	Get partition size.
Constraint	N/A
Parameters	@handle: fmd partition handle. @retsize: returned partition size.
Return value	Success return zero.

33.8 fmd_get_erase_size

Prototype	int fmd_get_erase_size(u32 handle, u32 *retsize)
Function	Get minimal erase size.
Constraint	N/A
Parameters	@handle: fmd partition handle. @retsize: returned erase size.
Return value	Success return zero.

34 Flash translation layer

Brief instruction

Flash translation layer(ftl) operate interface.

Header file	Location

AICXTEK Confidential

ftlblock.h	drivers\include
ftlrandom.h	drivers\include

34.1 ftlblock_open

Prototype	int ftlblock_open(char *name, u32 *handle)
Function	Open ftl partition by name, and return handle if success.
Constraint	N/A
Parameters	@name: ftl partition name. @handle: ftl partition handle.
Return value	Open success return zero.

34.2 ftlblock_close

Prototype	int ftlblock_close(u32 handle)
Function	Close ftl partition
Constraint	N/A
Parameters	@handle: ftl partition handle.
Return value	Close success return zero.

34.3 ftlblock_read

Prototype	int ftlblock_read(u32 handle, u32 sector_id, u32 sector_num, u8 *buf)
Function	Read continuous blocks data from ftl partition.
Constraint	N/A
Parameters	@handle: ftl partition handle. @sector_id: the sector id start to read in partition. @sector_num: the sector number want to read. @buf: the buffer to store read data.
Return value	Read success return zero.

34.4 ftlblock_write

Prototype	int ftlblock_write(u32 handle, u32 sector_id, u32 sector_num, u8 *buf)
Function	Write continuous blocks data to ftl partition.

AICXTEK Confidential

Constraint	N/A
Parameters	@handle: ftl partition handle. @sector_id: the sector id start to write in partition. @sector_num: the sector number to be write. @buf: the buffer to store writed data.
Return value	Write success return zero.

34.5 ftlblock_get_sector_size

Prototype	int ftlblock_get_sector_size(u32 handle, u32 *retsize)
Function	Get a single sector size.
Constraint	N/A
Parameters	@handle: ftl partition handle. @retsize: returned sector size.
Return value	Success return zero.

34.6 ftlblock_get_sector_num

Prototype	int ftlblock_get_sector_num(u32 handle, u32 *retnum)
Function	Get sector number of a partition.
Constraint	N/A
Parameters	@handle: ftl partition handle. @retnum: returned sector number.
Return value	Success return zero.

34.7 ftlblock_get_partition_size

Prototype	int ftlblock_get_partition_size(u32 handle, u32 *retsize)
Function	Get partition size.
Constraint	N/A
Parameters	@handle: ftl partition handle. @retsize: returned partition size.
Return value	Success return zero.

34.8 ftlrandom_open

Prototype	int ftlrandom_open(char *name, u32 *handle)

AICXTEK Confidential

Function	Open ftl partition by name, and return handle if success.
Constraint	N/A
Parameters	@name: ftl partition name. @handle: ftl partition handle.
Return value	Open success return zero.

34.9 ftlrandom_close

Prototype	int ftlrandom_close(u32 handle)
Function	Close ftl partition
Constraint	N/A
Parameters	@handle: ftl partition handle.
Return value	Close success return zero.

34.10 ftlrandom_read

Prototype	int ftlrandom_read(u32 handle, u32 offset, u32 size, u8 *buf)
Function	Read data from random address in ftl partition.
Constraint	The specified offset and size must not be out of partition range.
Parameters	@handle: ftl partition handle. @offset: the address start to read in partition. @size: the size want to read. @buf: the buffer to store read data.
Return value	Read success return zero.

34.11 ftlrandom_write

Prototype	int ftlrandom_write(u32 handle, u32 offset, u32 size, u8 *buf)
Function	Write data to random address in ftl partition.
Constraint	The specified offset and size must not be out of partition range.
Parameters	@handle: ftl partition handle. @offset: the address start to write in partition. @size: the size want to write. @buf: the buffer to store writed data.

AICXTEK Confidential

Return value	Write success return zero.

34.12 ftlrandom_get_sector_size

Prototype	int ftlrandom_get_sector_size(u32 handle, u32 *retsize)
Function	Get a single sector size.
Constraint	N/A
Parameters	@handle: ftl partition handle. @retsize: returned sector size.
Return value	Success return zero.

34.13 ftlrandom_get_sector_num

Prototype	int ftlrandom_get_sector_num(u32 handle, u32 *retnum)
Function	Get sector number of a partition.
Constraint	N/A
Parameters	@handle: ftl partition handle. @retnum: returned sector number.
Return value	Success return zero.

34.14 ftlrandom_get_partition_size

Prototype	int ftlrandom_get_partition_size(u32 handle, u32 *retsize)
Function	Get partition size.
Constraint	N/A
Parameters	@handle: ftl partition handle. @retsize: returned partition size.
Return value	Success return zero.

35 PowerSupply

Brief instruction

Power supply, SoC battery & charger interface.

35.1 aic_battery_monitor_start

Prototype	void aic_battery_monitor_start(void)

AICXTEK Confidential

Function	start battery monitor task.
Constraint	N/A
Parameters	N/A
Return value	N/A

35.2 aic_battery_monitor_stop

Prototype	void aic_battery_monitor_stop(void)
Function	Stop battery monitor task.
Constraint	N/A
Parameters	N/A
Return value	N/A

35.3 aic_pmic_charger_enable

Prototype	void aic_pmic_charger_enable(bool enabled)
Function	Start/Stop charge.
Constraint	N/A
Parameters	@ enabled: if true enable charging else disable charging.
Return value	N/A

35.4 get_psy_info

Prototype	void get_psy_info(struct psy_info *psy_info);
Function	Get battery info & charger status
Constraint	N/A
Parameters	@psy_info:point to battery&charge info.
Return value	Success return zero.

35.5 psy_changed_notifier_register

Prototype	int psy_changed_notifier_register(char *noticer_name, psy_noticer_call noticer_call);
Function	If power supply informantion has changed(battery cap\charge status\usb plugin\plug-out), system will notify registrant of this api.
Constraint	N/A
Parameters	@noticer_name:noticer name,user freedom to name. @noticer_call: user register callback, system will transfer power supply info to this call.
Return value	Success return zero.

AICXTEK Confidential

35.6 psy_changed_notifier_unregister

Prototype	int psy_changed_notifier_unregister(char *noticer_name);
Function	Unregister power supply notifier.
Constraint	N/A
Parameters	@noticer_name:noticer name, user already register in power supply notify list.
Return value	Success return zero.

36 LED

37 Lamp

Brief instruction

Lamp hardware driver operate interface.

Header file	Location
lamp.h	drivers\include

37.1 lamp_enabled

Prototype	int lamp_enabled(bool enabled);
Function	Switch light of lamp.
Constraint	should use in thread context.
Parameters	@enabled: switch lamp status true:light up, false:light off
Return value	Returns zero if the switch lamp status set successful, else a negative errno code.

37.2 lamp_is_enabled

Prototype	int lamp_is_enabled(void);
Function	Get current switch lamp status.
Constraint	should use in thread context.
Parameters	NA
Return value	Returns positive if the switch lamp has requested that the device be enabled. Zero if it hasn't , else a negative errno code.

37.3 lamp_set_current

AICXTEK Confidential

Prototype	int lamp_set_current(u32 current_uA);
Function	Set switch lamp output current.
Constraint	should use in thread context.
Parameters	@current_uA: required current in uA
Return value	Returns zero if the switch lamp current set successful, else a negative errno code.

37.4 lamp_get_current

Prototype	int lamp_get_current(u32 *current_uA);
Function	Set switch lamp output current.
Constraint	should use in thread context.
Parameters	@current_uA: point to current switch lamp current in uA.
Return value	Returns zero if the switch lamp current get successful, else a negative errno code.

38 Can

Brief instruction

Can driver operate interface.

Header file	Location
can_drv.h	drivers\include

38.1 can_init

Prototype	int can_init(can_dev *pdev);
Function	Initialize can drv.
Constraint	should use in thread context.
Parameters	@pdev: can dev initialize parameter
Return value	Returns zero if can initialize successful, else a negative errno code.

38.2 can_recv

Prototype	int can_recv(can_dev *pdev, can_frame *cf);
Function	receive message
Constraint	should use in thread context.
Parameters	@pdev: can dev @cf: buffer to store received message

AICXTEK Confidential

Return value	Returns zero if can receive successful, else a negative errno code.

38.3 can_send

Prototype	int can_send(can_dev *pdev, can_frame *cf);
Function	use STB to send message.
Constraint	should use in thread context.
Parameters	@pdev: can dev @cf: message to send
Return value	Returns zero if can send successful, else a negative errno code.

38.4 can_send_high_priority

Prototype	int can_send_high_priority (can_dev *pdev, can_frame *cf);
Function	use PTB to send message, it will be processed in high-priority than STB.
Constraint	should use in thread context.
Parameters	@pdev: can dev @cf: message to send
Return value	Returns zero if can send successful, else a negative errno code.

38.5 can_set_filter

Prototype	int can_set_filter(can_dev *pdev, can_filter_cfg *config);
Function	set can filter.
Constraint	should use in thread context.
Parameters	@pdev: can dev @config: can filter config parameter
Return value	Returns zero if can receive successful, else a negative errno code.

38.6 can_set_bit_timing

Prototype	int can_set_bit_timing(can_dev *pdev, u32 baudrate);
Function	set can bit timing.

AICXTEK Confidential

Constraint	should use in thread context.
Parameters	@pdev: can dev @baudrate: baudrate parameter
Return value	Returns zero if can receive successful, else a negative errno code.

AICXTEK Confidential